

南开大学

本科生毕业论文（设计）

中文题目： 支持隐私保护的机器学习组件优化探索

外文题目： Exploration on Optimization for components of
Privacy-Preserving Machines Learning

学 号： 2113069

姓 名： 冯宇好

年 级： 2021 级

专 业： 计算机科学与技术

系 别： 计算机科学与技术

学 院： 计算机学院

指导教师： 苏明 副研究员

完成日期： 2025 年 5 月

关于南开大学本科生毕业论文（设计）的声明

本人郑重声明：所呈交的学位论文，是本人在指导教师指导下，进行研究工作所取得的成果。除文中已经注明引用的内容外，本学位论文的研究成果不包含任何他人创作的、已公开发表或没有公开发表的作品内容。对本论文所涉及的研究工作做出贡献的其他个人和集体，均已在文中以明确方式标明。本学位论文原创性声明的法律责任由本人承担。

学位论文作者签名：冯宇好

2025年 5月 18日

本人声明：该学位论文是本人指导学生完成的研究成果，已经审阅过论文的全部内容，并能够保证题目、关键词、摘要部分中英文内容的一致性和准确性。

学位论文指导教师签名：苏明

2025年 5月 18日

摘 要

本文在卷积神经网络 (Convolutional Neural Network, CNN) 的基础上, 实现数据集 MINIST 的分类 CNN。具体地, 本文首先训练出具有高准确率的明文 CNN, 然后基于安全多方计算 (Secure Multi-Party Computation, SMPC) 技术, 实现了用于推理的密文 CNN, 最后分别对本文优化方案与多项式逼近方案进行对比实验, 并从运行效率、精确度和实用性三个方面进行对比分析。

在多项式逼近方案中, 利用同态加密技术, 实现卷积层和全连接层等线性层。对于非线性层 ReLU 层, 先利用多项式函数逼近 ReLU 函数, 再结合同态加密技术进行实现。同时对测试用例进行批处理, 避免卷积运算和矩阵乘法运算的旋转操作, 从而提高密文 CNN 的吞吐量与运行效率。但此方案具有严重的精度损失和数据溢出等问题。

本文优化方案采用同样的思路实现线性层。但在实现 ReLU 层时, 首先基于姚氏百万富翁问题, 利用 SMPC 技术实现其导函数 DReLU, 然后进一步实现 ReLU 函数, 避免了逼近操作的误差, 从而解决了精度损失问题。接着本文对线性层和 ReLU 层进行联合优化, 消除了旋转操作, 并减少了通信开销, 从而提高密文 CNN 的运行效率。最后本文实现了环上的除法协议, 使得当中间结果膨胀过大时, 可以利用除法协议截断中间结果, 从而解决数据溢出问题。结合上述所有优化, 本文实现了具有高精确度和高运行效率的密文 CNN。

关键词: MLaaS; 安全多方计算; 卷积神经网络; 联合优化; MINIST

Abstract

This thesis implements a Convolutional Neural Network (CNN) for classifying the MNIST dataset. Specifically, we first train a CNN on plaintext with high accuracy. Then, based on Secure Multi-Party Computation (SMPC) techniques, we construct an CNN on ciphertext for inference. Finally, we conduct comparative experiments between our optimized scheme and the polynomial approximation scheme, analyzing their performance in terms of runtime efficiency, accuracy, and practicality.

In the polynomial approximation scheme, homomorphic encryption is used to implement linear layers such as convolutional and fully connected layers. For the nonlinear ReLU layer, a polynomial function is first employed to approximate the ReLU function, which is then implemented using homomorphic encryption. Additionally, test cases are processed in batches to eliminate rotation operations in convolution and matrix multiplication, thereby improving the throughput of the encrypted CNN and enhancing its runtime efficiency. However, this approach suffers from significant accuracy loss and data overflow issues.

Our optimized scheme follows the same methodology for implementing linear layers. However, for the ReLU layer, this thesis first implement its derivative function (DReLU) using SMPC techniques based on Yao's Millionaires' Problem, then further construct the ReLU function itself. This avoids approximation errors and resolves the accuracy loss problem. Next, this thesis jointly optimize the linear and ReLU layers, eliminating rotation operations and reducing communication overhead, thereby improving the runtime efficiency of the encrypted CNN. Finally, we implement a division protocol over rings, which truncates intermediate results when they expand excessively, solving the data overflow issue. By integrating all these optimizations, we achieve an encrypted CNN with high accuracy and improved runtime efficiency.

Key Words: MLaaS;secure multi-party computation,convolutional neural network;allied optimization;MNIST

目 录

摘要		I
Abstract		II
目录		III
第一章 绪论		1
第一节 研究背景与挑战		1
第二节 主要工作与贡献		1
第三节 论文的组织结构		2
第二章 背景知识和相关工作		3
第一节 数学记号		3
第二节 卷积神经网络		3
第三节 定点数编码		4
第四节 同态加密		5
第五节 不经意传输		6
2.5.1 $\binom{2}{1}$ -OT 协议		6
2.5.2 $\binom{n}{1}$ -OT 协议		8
第六节 加性秘密共享		9
2.6.1 2 选 2 加性秘密共享协议		9
2.6.2 Beaver Triple 生成协议		10
第七节 姚氏百万富翁问题		11
第三章 密文网络实现方法及其优化		13
第一节 姚氏百万富翁问题的优化协议		13
第二节 计算 ReLU 函数的相关协议		15
第三节 线性层与非线性层的联合优化		18
3.3.1 $f_r(f_c(x))$ 的展开式		18
3.3.2 无旋卷积协议		19
3.3.3 $f_c(f_r(x))$ 协议		21
3.3.4 $f_w(f_c(x))$ 的展开式		22
3.3.5 无旋矩阵乘法协议		23
3.3.6 $f_w(f_r(x))$ 协议		24
3.3.7 $f_c(f_r(x))$ 和 $f_w(f_r(x))$ 协议的优化		25
第四节 共享除法协议		25

第五节 $\binom{n}{1}$ -OT 协议的优化	27
第四章 实验与分析	30
第一节 实验环境	30
第二节 明文卷积神经网络	30
第三节 密文卷积神经网络	31
4.3.1 加密参数的设置	31
4.3.2 密文网络的实现	32
4.3.3 截断操作的引入	33
4.3.4 $\binom{n}{1}$ -OT 协议的选择	33
第四节 性能对比分析	34
4.4.1 运行效率	34
4.4.2 精确度	36
4.4.3 实用性	37
第五章 总结与展望	39
参考文献	40
致 谢	XLIII

第一章 绪论

第一节 研究背景与挑战

机器学习即服务 (Machine Learning as a Service, MLaaS), 即服务器将机器学习算法部署到云平台, 使得用户能够上传数据, 并利用服务器提供的计算资源和模型参数进行推理。当 MLaaS 应用于金融和医疗等领域时, 参与方提出了如下的隐私保护需求: 用户希望个人数据不会泄露给服务器, 服务器希望模型参数不会被用户窃取。除此之外, MLaaS 还需要具备高运行效率和高精确度。目前实现满足上述需求的 MLaaS 是一件具有挑战性的任务。

在实现卷积层和全连接层时, 多项式逼近方案采用了同态加密技术^[1,2], 使得线性运算能够在密文上正确执行, 从而保证了运算的安全性和准确性。同时批处理测试用例, 进而避免了卷积运算和矩阵乘法运算中的旋转操作, 从而提高了运行效率和吞吐率。但在实际应用中, 批处理测试用例必须使用固定个数的测试用例进行编码, 从而导致编码后的明文出现大量空白填充, 进而大幅降低运行效率。

在实现 ReLU 层时, 多项式逼近方案利用多项式函数逼近 ReLU 函数^[1,2], 并结合同态加密技术进行实现。此方案虽然能够模拟 ReLU 函数的功能, 但使用环上的多项式函数极可能导致数据膨胀, 从而造成大量数据溢出问题。除以上方案之外, 其它相关方案^[3] 利用不经意传输、混淆电路和秘密共享等安全多方计算技术实现 ReLU 函数。该方案成功实现具有高精确度的 ReLU 函数, 但仍未彻底解决数据溢出问题, 同时产生了大量计算开销和通信开销。

如何在保证安全性的前提下, 高效且精确地实现非批处理 MLaaS 是值得关注的问题。

第二节 主要工作与贡献

本文基于开源同态加密库 SEAL (Simple Encrypted Arithmetic Library) 和开源安全多方计算库 EMP (Efficient Multi-Party computation toolkit), 针对上一节中的挑战实现了支持隐私保护的批处理密文 CNN。具体实现思路为: 首先基

于同态加密技术实现线性层；然后基于安全多方计算技术实现 ReLU 层；接着为了提高运行效率，对线性层和相邻的 ReLU 层进行联合优化；最后实现了共享除法协议，解决数据膨胀问题。

本文的主要工作及贡献包括：

1. 基于不经意传输和加性秘密共享等技术，并结合分治思想实现了具有低时间开销的姚氏百万富翁问题协议。
2. 将计算 ReLU 函数转化成计算其导函数 DReLU，并结合姚氏百万富翁问题协议实现了 ReLU 层。
3. 将线性层和相邻的 ReLU 层进行联合优化，消除了线性层的旋转操作，以及从 DReLU 函数到 ReLU 函数所需的计算开销和通信开销。
4. 基于不经意传输实现了共享除法协议，利用共享除法协议对中间结果进行截断，解决了数据膨胀问题。
5. 进行测试实验探究加密参数、缩放因子和截断操作，对密文 CNN 的运行效率和精确度的影响。并从运行效率、精确度和实用性三个方面，将本文优化方案与多项式逼近方案进行对比实验。

第三节 论文的组织结构

本文共包括五个章节，其内容组织结构如下：

第一章，介绍本文的研究背景与挑战、主要工作及贡献和论文的组织结构。

第二章，介绍常用的数学记号、卷积神经网络、定点数编码、同态加密、不经意传输、加性秘密共享和姚氏百万富翁问题的理论知识以及一些国内外相关工作。

第三章，介绍密文网络的实现方法以及相关优化。

第四章，介绍实验环境、参数探究实验、以及对比实验的分析与讨论。

第五章，对本文工作进行总结，并讨论未来工作扩展和研究方向。

第二章 背景知识和相关工作

第一节 数学记号

在本文中定义如下数学记号： $\lceil \cdot \rceil$ 和 $\lfloor \cdot \rfloor$ 分别表示上取整函数和下取整函数；集合 $\mathbb{Z}_n = \mathbb{Z} \cap [-\lfloor n/2 \rfloor, \lfloor n/2 \rfloor]$ ，其中整数 $n > 2$ ，特别地 $\mathbb{Z}_2 = \{0, 1\}$ ；环 $\llbracket i \rrbracket = \{0, 1, \dots, i-1\}$ ，其中整数 $i > 0$ ； $\mathbf{1}\{\mathcal{I}\}$ 表示指示函数，当 $\mathcal{I}$ 为真时，函数值为 1，当 $\mathcal{I}$ 为假时，函数值为 0； $r \stackrel{\$}{\leftarrow} \mathcal{D}$ 表示从集合 $\mathcal{D}$ 中随机选出元素 r ；运算符 $+$, $-$, $\boxtimes$ 分别表示在明文域（或密文域）上的逐元素加法、减法和乘法运算；运算符 $*$ 表示矩阵的卷积运算；运算符 $\cdot$ 表示一般意义的点积运算，包括数乘、向量点积和矩阵乘法；运算符 $\oplus, \&$ 表示按位异或和按位与运算；运算符 $\parallel$ 表示比特串连接运算。

第二节 卷积神经网络

卷积神经网络（Convolutional Neural Network, CNN）^[4, 5] 是一种专门用于处理具有网格结构数据（如图像、视频、语音）的深度学习模型。其核心思想是通过局部连接、权值共享和层次化特征提取，高效地捕捉输入数据中的空间或时间局部模式。CNN 的结构被可视化为运算层的堆叠，每一层的输入来自上一层的输出，顶层的输入由用户提供，底层的输出为推理结果。常见的运算层类型有：

1. 卷积层（Convolutional Layer）：卷积运算 $f_c(\cdot)$ 由 $x \in \mathbb{R}^{C_i \times H_i \times W_i}$ 、卷积核 $k \in \mathbb{R}^{C_o \times C_i \times f_h \times f_h}$ 和步长 s 参与运算，输出 $y \in \mathbb{R}^{C_o \times H_o \times W_o}$ ，其中 C_i 为输入通道数， H_i 和 W_i 分别表示每个二维输入通道的高度和宽度， f_h 为每个二维卷积核的边长， C_o 为输出通道数， H_o 和 W_o 分别表示每个二维输出通道的高度和宽度。在运算过程中，输出通道的每个值，均由其通道数对应的 C_i 个卷积核，分别与 C_i 个输入通道的相应位置，做逐项乘法后求和得到。且每做一次求和，需要将相应位置移动 s 个单位距离，再计算下一个值。因此有等式 $H_o = (H_i - f_h)/s + 1$, $W_o = (W_i - f_h)/s + 1$ 。
2. 全连接层（Fully Connection Layer）：全连接运算 $f_w(\cdot)$ 由输入 $x \in \mathbb{R}^{N_i}$ 和权值矩阵 $w \in \mathbb{R}^{N_o \times N_i}$ 参与运算，输出 $y \in \mathbb{R}^{N_o}$ ，其中 N_i 和 N_o 分别为输入

向量的长度和输出向量的长度。在运算过程中，将权值矩阵和输入向量做矩阵乘法，即可得到输出向量。在全连接层之后，通常结合 Softmax 等函数，进行归一化。

3. 激活层 (Activation Layer): 激活运算 $f_r(\cdot)$ 使用特定的激活函数，处理输入通道的每个值，得到与输入通道具有相同结构的输出通道。本文使用的激活函数为 ReLU (Rectified Linear Unit) 函数，其定义如下

$$\text{ReLU}(x) = \max(x, 0), x \in \mathbb{R}, \quad (2.1)$$

并记其导函数为

$$\text{DReLU}(x) = \mathbf{1}\{x \geq 0\}, x \in \mathbb{R}. \quad (2.2)$$

4. 池化层 (Pooling Layer): 池化运算使用特定的池化函数，处理输入通道上固定大小窗口里的所有值。每次处理后，窗口移动固定步长，再进行下一次处理。常见的池化函数有均值函数和最大值函数。本文不涉及池化层的实验，在此不过多介绍。

第三节 定点数编码

在安全多方计算中，通常需要将浮点数编码为定点数，再将定点数编码为环或域上的整数，才能参与协议的计算。在本文中设置一个缩放因子 scale 和一个明文模数 p 。对于给定的浮点数 $x \in \mathbb{R}$ ，先将其编码为定点数 $\lfloor \text{scale} \cdot x \rfloor$ ，然后编码成环 $\llbracket p \rrbracket$ 上的整数 $\lfloor \text{scale} \cdot x \rfloor \bmod p$ ，其中 p 通常为奇素数。具体的编码和解码函数定义如下：

$$\text{encode}(x) = \lfloor \text{scale} \cdot x \rfloor \bmod p, x \in \mathbb{R}, \quad (2.3)$$

$$\text{decode}(y) = \begin{cases} y/\text{scale}, y \leq \lfloor p/2 \rfloor \wedge y \in \llbracket p \rrbracket, \\ (y - p)/\text{scale}, y > \lfloor p/2 \rfloor \wedge y \in \llbracket p \rrbracket. \end{cases} \quad (2.4)$$

在应用此编码方式，进行安全多方计算时，通常会出现以下问题：

1. 数据溢出：定点数的绝对值不能超过 $\lfloor p/2 \rfloor$ ，且在复杂的神经网络中，定点数表示下的中间结果的绝对值也不能超过 $\lfloor p/2 \rfloor$ 。否则，定点数被编码成环 $\llbracket p \rrbracket$ 上的整数后，无法通过上述解码函数，得到正确的原数据。

因此需要选取较大的明文模数 p 和适当大小的缩放因子 scale ，保证数据的精确度和正确性。

2. 数据膨胀：注意到对于任意 $x, y \in \mathbb{R}$ ，有

$$\begin{aligned} \text{encode}(x) \cdot \text{encode}(y) &= (\lfloor \text{scale} \cdot x \rfloor \cdot \lfloor \text{scale} \cdot y \rfloor) \bmod p \\ &\approx \lfloor \text{scale}^2 \cdot xy \rfloor \bmod p, \end{aligned} \quad (2.5)$$

从而有

$$\text{decode}(\text{encode}(x) \cdot \text{encode}(y)) \approx \text{scale} \cdot xy, \quad (2.6)$$

即每做一次环上的乘法，原结果都会膨胀大约 scale 倍。随着数据不断膨胀，极有可能导致数据溢出。因此需要在乘法运算后除以 scale ，使得解码后能够得到合适的结果，这种操作称为截断（Truncation）。

第四节 同态加密

同态加密（Homomorphic Encryption, HE）^[6,7] 是满足明文加法群与密文加法群同态、明文乘法群与密文乘法群同态的密码体制，即

$$\text{Dec}(sk, \text{Enc}(pk, x) + \text{Enc}(pk, y)) = x + y, \quad (2.7)$$

$$\text{Dec}(sk, \text{Enc}(pk, x) \boxtimes \text{Enc}(pk, y)) = x \boxtimes y, \quad (2.8)$$

其中 sk, pk 分别为私钥和公钥， $\text{Enc}(\cdot), \text{Dec}(\cdot)$ 分别为加密和解密运算。本文使用 SEAL 库中的完全同态加密方案 BFV^[8,9]，并用 p 表示明文模数， q 表示密文模数， N 表示明文多项式度数。该加密方案支持批量编码（BatchEncode），能将长度为 N 的向量 $\mathbf{x} = (x_0, x_1, \dots, x_{N-1}) \in \llbracket p \rrbracket^N$ 编码为一个明文多项式，并支持如下同态运算：

1. 同态加法（Homomorphic Addition, $+$ ）： $\text{Dec}(sk, [\mathbf{u}]_{pk} + [\mathbf{v}]_{pk}) = \mathbf{u} + \mathbf{v}$ 和 $\text{Dec}(sk, [\mathbf{u}]_{pk} + \mathbf{v}) = \mathbf{u} + \mathbf{v}$ 。
2. 同态减法（Homomorphic Subtraction, $-$ ）： $\text{Dec}(sk, [\mathbf{u}]_{pk} - [\mathbf{v}]_{pk}) = \mathbf{u} - \mathbf{v}$ 和 $\text{Dec}(sk, [\mathbf{u}]_{pk} - \mathbf{v}) = \mathbf{u} - \mathbf{v}$ 。
3. 同态乘法（Homomorphic Multiplication, $\boxtimes$ ）： $\text{Dec}(sk, [\mathbf{u}]_{pk} \boxtimes [\mathbf{v}]_{pk}) = \mathbf{u} \boxtimes \mathbf{v}$

和 $\text{Dec}(sk, [\mathbf{u}]_{pk} \boxtimes \mathbf{v}) = \mathbf{u} \boxtimes \mathbf{v}$ 。

4. 同态旋转 (Homomorphic Rotation, Rot): $\text{Dec}(sk, \text{Rot}([\mathbf{u}]_{pk}, l)) = \mathbf{u}_l$, 这里 $\mathbf{u}_l = (u_l, \dots, u_{N-1}, u_0, \dots, u_{l-1}) \in \llbracket p \rrbracket^N$ 和 $l \in \{0, 1, \dots, N-1\}$ 。

其中 $\mathbf{u} = (u_0, u_1, \dots, u_{N-1})$ 和 $\mathbf{v} = (v_0, v_1, \dots, v_{N-1})$, $[\cdot]_{pk}$ 表示使用公钥 pk 加密的密文。

在执行上述同态运算时, 会消耗解密器的噪声预算, 当噪声预算消耗尽时, 解密器无法正确解密。而噪声预算的消耗速度与明文模数 p 的大小正相关, 初始噪声预算的大小与密文模数 q 的大小正相关。同时明文多项式的度数 N 与密文模数 q 的上限正相关, 具体关系如表 2.1。因此选取合适的明文多项式度数、明文模数和密文模数, 是实现具有高精度和正确性的密文 CNN 的关键。

表 2.1 明文多项式度数与密文模数的最大比特数关系

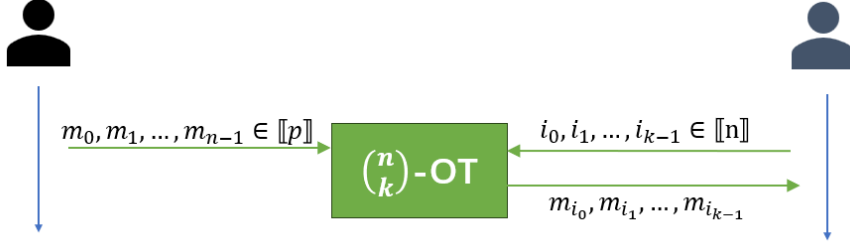
明文多项式度数	密文模数的最大比特数
1024	27
2048	54
4096	109
8192	218
16384	438
32768	881

第五节 不经意传输

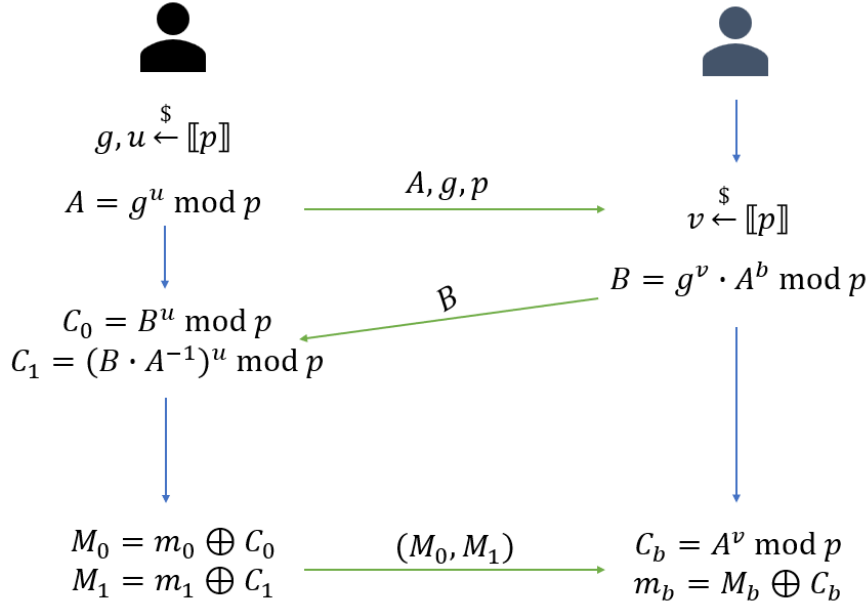
不经意传输 (Oblivious Transfer, OT)^[10] 可描述为如下问题: 设有两个参与方为发送方和接收方 (简记为 S 和 R), S 具有 n 个信息 $m_0, m_1, \dots, m_{n-1} \in \llbracket p \rrbracket$, R 想要获取其中下标为 $i_0, i_1, \dots, i_{k-1} \in \llbracket n \rrbracket$ 的 k 个信息, 其中 p 为素数。要求双方在不泄露任何信息的情况下, 完成信息传输。本文将该问题简记为 $\binom{n}{k}$ -OT, 且主要介绍 $\binom{2}{1}$ -OT 协议和 $\binom{n}{1}$ -OT 协议, 图 2.1 展示了 $\binom{n}{k}$ -OT 的抽象结构。

2.5.1 $\binom{2}{1}$ -OT 协议

本小节介绍一种基于离散对数难题 (Discrete Logarithm Problem, DLP) 的 $\binom{2}{1}$ -OT 协议^[11]。假设 S 持有 2 个信息 $m_0, m_1 \in \llbracket p \rrbracket$, R 想要获取其中下标为 $b \in \{0, 1\}$ 的信息。该协议流程如图 2.2 所示, 双方执行如下步骤:

图 2.1 $\binom{n}{k}$ -OT 示意图

1. S 从 $\llbracket p \rrbracket$ 中随机选取原根 g 和非零整数 u , 计算 $A = g^u \bmod p$, 并将 A, g, p 发送给 R。
2. R 从 $\llbracket p \rrbracket$ 中随机选取非零整数 v , 计算 $B = A^v \cdot g^v \bmod p$, 并将 B 发送给 S。此时基于离散对数难题, 即使 R 知道 A, g, p , 也难以计算出 u 。
3. S 计算 $C_0 = B^u \bmod p, C_1 = (B \cdot A^{-1})^u \bmod p$, 然后计算 $M_0 = m_0 \oplus C_0, M_1 = m_1 \oplus C_1$, 并将有序对 (M_0, M_1) 发送给 R。此时由于 R 引入了混淆项 v , S 因而无法根据 B 推断出 b 。
4. R 计算 $C_b = A^v \bmod p$, 然后获得消息 $m_b = M_b \oplus C_b$ 。此时由于 R 无法得知 u , 从而无法计算 A^u 或 A^{-u} , 进而无法通过 M_{1-b} , 获取另一个消息 m_{1-b} 。

图 2.2 $\binom{2}{1}$ -OT 协议流程图

本文使用 EMP 库中的 $\binom{2}{1}$ -OT 方案 INKP^[12], 该方案借助巧妙的矩阵变换,

在离线阶段执行少量上述 $\binom{2}{1}$ -OT 协议，生成大量关联的 $\binom{2}{1}$ -OT (Correlated Oblivious Transfer, COT) 的输入和输出，在在线阶段使用这些 COT，计算大量只包含异或运算的 $\binom{2}{1}$ -OT，从而减少了环上的幂运算开销。

2.5.2 $\binom{n}{1}$ -OT 协议

本小节基于 2.5.1 节，构造了一种时间复杂度为 n 次 $\binom{2}{1}$ -OT 的简单 $\binom{n}{1}$ -OT 协议。假设 S 持有 n 个信息 $m_0, m_1, \dots, m_{n-1} \in \llbracket p \rrbracket$ ， R 想要获取其中下标为 $c \in \llbracket n \rrbracket$ 的信息。该协议流程如图 2.3 所示，双方执行如下步骤：

1. S 随机选取 $k_i \in \llbracket p \rrbracket (i = 0, 1, \dots, n-1)$ 。
2. S 和 R 进行 n 次 $\binom{2}{1}$ -OT 协议。在第 $i+1$ 次协议中， S 输入 $k_0 \oplus k_1 \oplus \dots \oplus k_{i-1} \oplus m_i$ 和 k_i 。当 $i = c$ 时， R 输入 0；当 $i \neq c$ 时， R 输入 1。
3. R 通过第 1 次到第 c 次 $\binom{2}{1}$ -OT 获得 $k_0, k_1, \dots, k_{c-1}$ ，第 $c+1$ 次获得 $k_0 \oplus k_1 \oplus \dots \oplus k_{c-1} \oplus m_c$ ，进而求异或和可得到 m_c 。

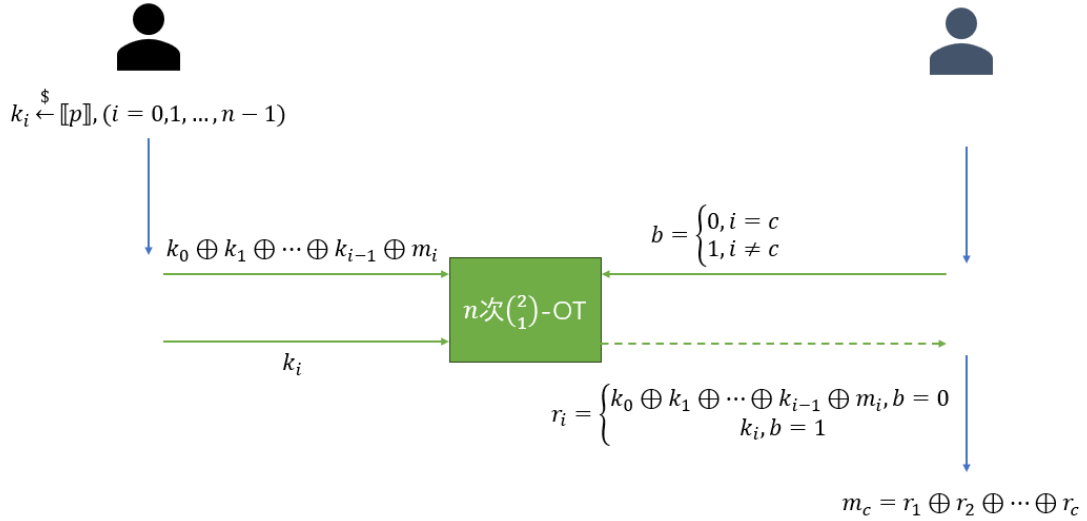


图 2.3 $\binom{n}{1}$ -OT 协议流程图

基于上述 $\binom{n}{1}$ -OT 协议，可以安全计算如下 $n+m$ 元函数。设 $n+m$ 元函数 $f(x_1, x_2, \dots, x_n; w_1, w_2, \dots, w_m) : \llbracket 2^l \rrbracket^{n+m} \rightarrow \llbracket 2^l \rrbracket$ ，其中 $l \in \mathbb{N}$ 为比特串长度上限， $x_1, x_2, \dots, x_n \in \llbracket 2^l \rrbracket$ 为用户输入， $w_1, w_2, \dots, w_m$ 为服务器参数。服务器预计算出 $(x_1, x_2, \dots, x_n) = (0, 0, \dots, 0), \dots, (2^l - 1, 2^l - 1, \dots, 2^l - 1)$ 的函数值，记为 $f_0, f_1, \dots, f_{2^{nl}-1}$ ，并作为 $\binom{2^{nl}}{1}$ -OT 的发送方输入；用户设置 $c = x_1 || x_2 || \dots || x_n$ 作为 $\binom{2^{nl}}{1}$ -OT 的接收方输入，即可安全地获得相应的函数值。

第六节 加性秘密共享

加性秘密共享 (Additive Secret Sharing, ASS)^[13] 可描述为如下问题: 秘密持有方将原秘密 $s \in \llbracket n \rrbracket$ 在参与方 $P_1, P_2, \dots, P_m$ 中共享, 每个参与方获得秘密份额 $s_i \in \llbracket n \rrbracket (i = 1, 2, \dots, m)$, 满足 $s_1 + s_2 + \dots + s_m = s$, 其中 $n \in \mathbb{N}$ 。要求关于 s 的运算可以等价于各参与方之间的运算, 且各参与方不会泄露各自的秘密份额。

2.6.1 2 选 2 加性秘密共享协议

2 选 2 加性秘密共享 (2-outof-2 Additive Secret Sharing) 可描述为如下子协议:

1. 秘密分发 $\text{Share}^n(x)$: 秘密持有方将秘密 $x \in \llbracket n \rrbracket$, 分发给参与方 P_0 和 P_1 , 双方分别持有秘密份额 $\langle x \rangle_0^n$ 和 $\langle x \rangle_1^n$, 满足 $\langle x \rangle_0^n + \langle x \rangle_1^n = x$ 。
2. 秘密重组 $\text{Reconst}^n(\langle x \rangle_0^n, \langle x \rangle_1^n)$: 参与方 $P_i (i = 0, 1)$ 将 $\langle x \rangle_i^n$ 发送给参与方 P_{1-i} , 对方接收到信息后, 计算 $x = \langle x \rangle_i^n + \langle x \rangle_{1-i}^n$ 。
3. 共享加法: 给定秘密 $x, y \in \llbracket n \rrbracket$ 的秘密共享, 计算加法运算 $z = x + y$, 该运算等价于参与方 $P_i (i = 0, 1)$ 分别本地计算 $\langle z \rangle_i^n = \langle x \rangle_i^n + \langle y \rangle_i^n$ 。
4. 共享乘法: 给定秘密 $x, y \in \llbracket n \rrbracket$ 的秘密共享, 计算乘法运算 $z = x \cdot y$, 该运算的实现细节在下一段介绍。

共享乘法不能简单地通过参与方本地计算实现, 而需要设计更加复杂的协议, 才能精确计算共享乘法。Beaver 提出了如下的实现思路^[14]: 先利用代数变形将乘法函数展开, 然后把展开式的各项通过本地计算和通信交互分配给参与双方, 最终实现共享乘法。该协议流程如图 2.4 所示, 具体协议如下:

- 离线阶段: 参与方 P_0 和 P_1 共同生成一个三元组 (a, b, c) 的秘密共享, 其中 $a, b, c \in \llbracket n \rrbracket, c = a \cdot b$, 双方分别持有秘密份额 $(\langle a \rangle_0^n, \langle b \rangle_0^n, \langle c \rangle_0^n)$ 和 $(\langle a \rangle_1^n, \langle b \rangle_1^n, \langle c \rangle_1^n)$, 这样的三元组称为 Beaver Triple^[14]。关于 Beaver Triple 的生成方法在下一小节介绍。
- 在线阶段: 注意到有如下展开式

$$xy = (x - a)(y - b) + a(y - b) + b(x - a) + ab, \quad (2.9)$$

且参与方 $P_i (i = 0, 1)$ 分别持有秘密份额 $(\langle a \rangle_i^n, \langle b \rangle_i^n, \langle c \rangle_i^n, \langle x \rangle_i^n, \langle y \rangle_i^n)$, 双方执行如下协议, 即可得到乘法运算结果的秘密共享:

1. $P_i (i = 0, 1)$ 本地计算 $\langle e \rangle_i^n = \langle x \rangle_i^n - \langle a \rangle_i^n, \langle f \rangle_i^n = \langle y \rangle_i^n - \langle b \rangle_i^n$ 。

2. $P_i (i = 0, 1)$ 执行 $\text{Reconst}^n(\langle e \rangle_0^n, \langle e \rangle_1^n)$ 和 $\text{Reconst}^n(\langle f \rangle_0^n, \langle f \rangle_1^n)$, 恢复出秘密 e, f 。
3. $P_i (i = 0, 1)$ 本地计算 $\langle z \rangle_i^n = \langle c \rangle_i^n + \langle b \rangle_i^n \cdot e + \langle a \rangle_i^n \cdot f$ 。
4. P_0 计算 $\langle z \rangle_0^n = \langle z \rangle_0^n + ef$ 。

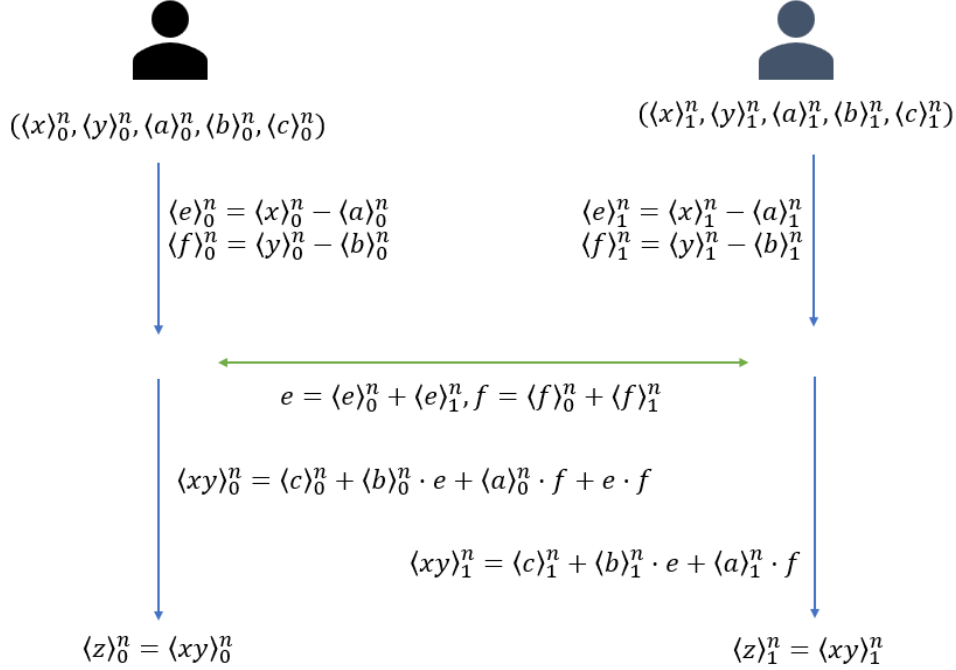


图 2.4 共享乘法的在线阶段协议流程图

本文主要考虑在环 $\mathbb{Z}_2$ 和环 $[n]$ 上的秘密共享，其中 n 为大于 2 的正整数，分别记为 $x = \langle x \rangle_0^B + \langle x \rangle_1^B$ 和 $x = \langle x \rangle_0^n + \langle x \rangle_1^n$ ，并分别称为布尔秘密共享和算数秘密共享。

2.6.2 Beaver Triple 生成协议

传统的 Beaver Triple 生成协议^[14] 基于同态加密技术，而本文只考虑在环 $\mathbb{Z}_2$ 上的布尔秘密共享，因此采用计算开销更小的不经意传输实现。下面通过 $\binom{4}{1}$ -OT 构造一组 Beaver triple $(\langle a \rangle_i^B, \langle b \rangle_i^B, \langle c \rangle_i^B) (i = 0, 1)$ ，注意到 $\langle a \rangle_i^B$ 和 $\langle b \rangle_i^B$ 可以在本地随机生成，因此只需计算出 $\langle c \rangle_i^B$ 即可。结合 $\binom{n}{1}$ -OT 实现安全计算 $n + m$ 元函数的思路，可以将 $\langle a \rangle_1^B$ 和 $\langle b \rangle_1^B$ 看作参与方 P_1 的输入， $\langle a \rangle_0^B$ 、 $\langle b \rangle_0^B$ 和随机生成的 $\langle c \rangle_0^B$ 作为参与方 P_0 的参数，从而参与方 P_1 可安全地计算出 $\langle c \rangle_1^B$ 。该协议流程如图 2.5 所示，具体协议如下：

1. $P_i (i = 0, 1)$ 本地随机生成秘密份额 $\langle a \rangle_i^B, \langle b \rangle_i^B \xleftarrow{\$} \{0, 1\}$;

2. P_1 作为接收方设置选择比特串 $\langle a \rangle_1^B || \langle b \rangle_1^B$ 作为 $\binom{4}{1}$ -OT₁ 的输入；
3. P_0 作为发送方，随机生成比特 $r \xleftarrow{\$} \{0,1\}$ 。对于所有 $j \in \{0,1\}^2$ ，设置第 j 组消息比特为 $r \oplus ((j_1 \oplus \langle a \rangle_0^B) \wedge (j_2 \oplus \langle b \rangle_0^B))$ 作为 $\binom{4}{1}$ -OT₁ 的输入，其中 $j = j_1 || j_2$ 。
4. P_0 设置 $\langle c \rangle_0^B = r$ ， P_1 设置 $\binom{4}{1}$ -OT₁ 的输出为 $\langle c \rangle_1^B$ 。

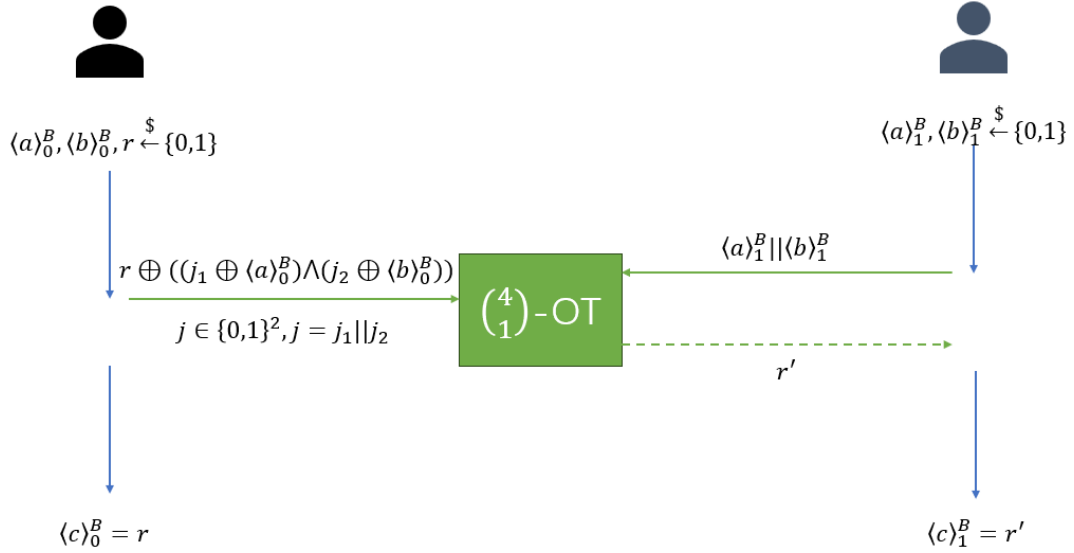


图 2.5 Beaver Triple 生成协议流程图

第七节 姚氏百万富翁问题

姚氏百万富翁问题 (Yao's Millionaires' problem, YMP) ^[15, 16] 可描述为：假设 Alice 有 x 百万的财产，Bob 有 y 百万的财产，其中 $x, y \in \llbracket n \rrbracket, n \in \mathbb{N}$ 。两人想在不泄露财产数值的情况下，比较财产大小。

其抽象的解决协议可描述为：

1. Alice 准备 n 个箱子，并编号为 $0, 1, 2, \dots, n-1$ 。然后在编号为 $0, 1, \dots, x$ 的箱子里放入纸条”no”，在其余的箱子里放入纸条”yes”。然后锁上所有箱子，并把它们交给 Bob。
2. Bob 选择编号为 y 的箱子，并将箱子的编号撕毁，然后把该箱子交给 Alice。最后把其余箱子全部销毁。
3. Alice 打开 Bob 发来的箱子，并公开其中的纸条信息。若纸条是”yes”，则表示 $x < y$ ，否则表示 $x \geq y$ 。

目前有利用 $\binom{n}{1}$ -OT 实现上述协议的方案^[17]。更简单的方案可以描述为: Alice 作为发送方将 $m_0, m_1, \dots, m_x$ 置 1, 将 $m_{x+1}, \dots, m_{n-1}$ 置 0, 并将 $m_i, (i = 0, 1, \dots, n-1)$ 作为 $\binom{n}{1}$ -OT 的输入; Bob 作为接收方设置 $b = y$, 并将 b 作为 $\binom{n}{1}$ -OT 的输入, 接受信息 m_b , 并将 m_b 的值向 Alice 公开, 若 $m_b = 0$ 则有 $x < y$, 否则有 $x \geq y$ 。进一步, 结合加性秘密共享, Alice 引入混淆项 $r \in \mathbb{Z}_2$, 并将原输入 m_i 改为 $r \oplus m_i$; Bob 保持原输入不变, 获得输出 s 。从而双方在不泄露财产数值的情况下, 获得了 $\mathbf{1}\{x < y\}$ 的布尔秘密共享 $\langle \mathbf{1}\{x < y\} \rangle_0^B = r, \langle \mathbf{1}\{x < y\} \rangle_1^B = s$ 。

在本文的实验中, x, y 的比特串长度 L 通常为 64, 而 $n = 2^L$ 是一个相当大的数, 因此时间复杂度高达 2^L 次 $\binom{2}{1}$ -OT。在下一章中, 将结合分治思想优化其时间开销。

第三章 密文网络实现方法及其优化

第一节 姚氏百万富翁问题的优化协议

基于第二章的第六节与第七节，本节提出复杂度更低的优化协议。问题描述为：设参与方 P_0 持有秘密 x ，参与方 P_1 持有秘密 y ，其中 $x, y \in \llbracket p \rrbracket$ ， p 为素数。要求在不泄露秘密的情况下，使得双方获得 $\mathbf{1}\{x < y\}$ 的布尔秘密共享。在本文的实验中，数据均采用 64 比特的无符号整型变量存储，因此以下在环 $\llbracket 2^L \rrbracket$ 中实现 YMP 协议，其中 $2^L > p$ 。

首先注意到 $\mathbf{1}\{x < y\}$ 有如下展开式^[18]

$$\mathbf{1}\{x < y\} = \mathbf{1}\{x_1 < y_1\} \oplus (\mathbf{1}\{x_1 = y_1\} \wedge \mathbf{1}\{x_2 < y_2\}), \quad (3.1)$$

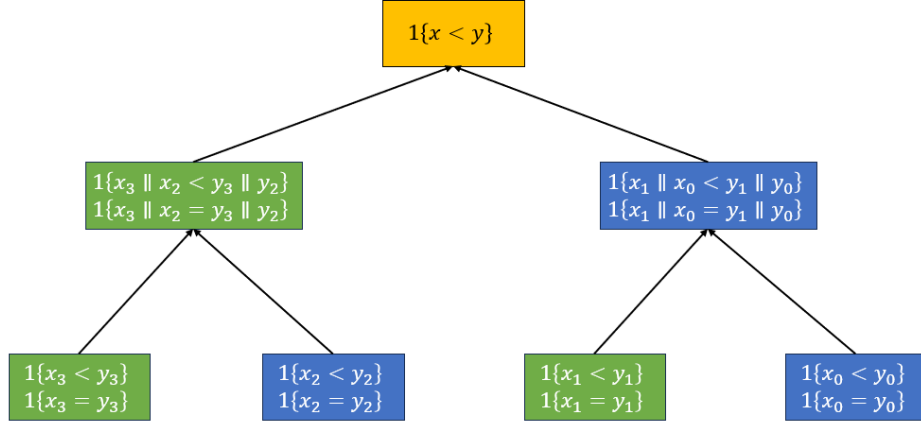
其中 $x = x_1 || x_2$, $y = y_1 || y_2$ ，且 x_1, x_2, y_1, y_2 均为长度为 $L/2$ 的比特串。基于分治思想，可以将计算 $\mathbf{1}\{x < y\}$ 转化为计算规模更小的 $\mathbf{1}\{x_1 < y_1\}$ 、 $\mathbf{1}\{x_2 < y_2\}$ 、 $\mathbf{1}\{x_1 = y_1\}$ 和 $\mathbf{1}\{x_2 = y_2\}$ ，再结合如下展开式

$$\mathbf{1}\{x = y\} = \mathbf{1}\{x_1 = y_1\} \wedge \mathbf{1}\{x_2 = y_2\}, \quad (3.2)$$

可以继续对 x_1, x_2, y_1, y_2 进行分治。综上，可以将 x, y 分割成长度为 m 的子比特串，即 $x = x_{q-1} || x_{q-2} || \dots || x_0$, $y = y_{q-1} || y_{q-2} || \dots || y_0$ ，在此先考虑 $m|L$ 且 $q = 2^\alpha, \alpha \in \mathbb{N}$ 的情况。对于每个 $\mathbf{1}\{x_i < y_i\}$ 和 $\mathbf{1}\{x_i = y_i\}$ 可利用 $\binom{2^m}{1}$ -OT 计算其布尔秘密共享，然后结合环 $\mathbb{Z}_2$ 上的共享乘法协议，可以自下而上地实现上述分治计算，整个计算过程为一棵满二叉树的结构，最终获得 $\mathbf{1}\{x < y\}$ 的布尔秘密共享 $\langle \mathbf{1}\{x < y\} \rangle_0^B$ 和 $\langle \mathbf{1}\{x < y\} \rangle_1^B$ ，图 3.1 展示了 $q = 4$ 的分治结构图。

在此以 $x = 10 = (1010)_2, y = 11 = (1011)_2$ 的情况为例进行说明，取 $m = 2, q = 2$ ，此时 $x_1 = 2, x_0 = 2$ 以及 $y_1 = 2, y_0 = 3$ 。结合第二章第七节的解决方案，不妨设经过不经意传输协议后， P_0 持有秘密份额：

$$\begin{aligned} \langle \mathbf{1}\{x_1 < y_1\} \rangle_0^B &= 1, \langle \mathbf{1}\{x_1 = y_1\} \rangle_0^B = 1, \\ \langle \mathbf{1}\{x_0 < y_0\} \rangle_0^B &= 0, \langle \mathbf{1}\{x_0 = y_0\} \rangle_0^B = 1. \end{aligned}$$


 图 3.1 $q=4$ 的 YMP 协议流程图

P_1 持有秘密份额:

$$\begin{aligned} \langle \mathbf{1}\{x_1 < y_1\} \rangle_1^B &= 1, \langle \mathbf{1}\{x_1 = y_1\} \rangle_1^B = 0, \\ \langle \mathbf{1}\{x_0 < y_0\} \rangle_1^B &= 1, \langle \mathbf{1}\{x_0 = y_0\} \rangle_1^B = 1. \end{aligned}$$

结合共享乘法协议, P_0 和 P_1 做关于秘密 $\mathbf{1}\{x_1 = y_1\}$ 和秘密 $\mathbf{1}\{x_0 < y_0\}$ 的乘法, 得到 $\mathbf{1}\{x_1 = y_1\} \wedge \mathbf{1}\{x_0 < y_0\}$ 的布尔秘密共享, 不妨设为

$$\begin{aligned} \langle \mathbf{1}\{x_1 = y_1\} \wedge \mathbf{1}\{x_0 < y_0\} \rangle_0^B &= 0, \\ \langle \mathbf{1}\{x_1 = y_1\} \wedge \mathbf{1}\{x_0 < y_0\} \rangle_1^B &= 1. \end{aligned}$$

最后双方在本地执行关于秘密 $\mathbf{1}\{x_1 < y_1\}$ 和秘密 $\mathbf{1}\{x_1 = y_1\} \wedge \mathbf{1}\{x_0 < y_0\}$ 的加法, 得到 $\mathbf{1}\{x < y\}$ 的布尔秘密共享:

$$\begin{aligned} \langle \mathbf{1}\{x < y\} \rangle_0^B &= \langle \mathbf{1}\{x_1 < y_1\} \rangle_0^B \oplus \langle \mathbf{1}\{x_1 = y_1\} \wedge \mathbf{1}\{x_0 < y_0\} \rangle_0^B = 1, \\ \langle \mathbf{1}\{x < y\} \rangle_1^B &= \langle \mathbf{1}\{x_1 < y_1\} \rangle_1^B \oplus \langle \mathbf{1}\{x_1 = y_1\} \wedge \mathbf{1}\{x_0 < y_0\} \rangle_1^B = 0. \end{aligned}$$

类似地可以计算 $\mathbf{1}\{x = y\}$ 的布尔秘密共享。

下面进行复杂度分析, 每个 $\mathbf{1}\{x_i < y_i\}$ 和 $\mathbf{1}\{x_i = y_i\}$ 需要进行 2^m 次 $\binom{2}{1}$ -OT, 因此分治算法的底层计算一共需要执行 $q \cdot 2^{m+1}$ 次 $\binom{2}{1}$ -OT。在分治的递归阶段, 由满二叉树的性质可得, 需要进行 $q-1$ 次递归合并, 每次递归合并中需要计算 (3.1) 式和 (3.2) 式, 即需要执行 2 次共享乘法协议。而每次共享乘法协议

需要执行一次 $\binom{4}{1}$ -OT, 即 4 次 $\binom{2}{1}$ -OT, 从而递归阶段一共进行了 $(q-1) \cdot 2^3$ 次 $\binom{2}{1}$ -OT。综上, 整个 YMP 协议一共执行了 $q \cdot 2^{m+1} + (q-1) \cdot 2^3$ 次 $\binom{2}{1}$ -OT。当 $L=64, m=4, q=16$ 时, 只需要进行 632 次 $\binom{2}{1}$ -OT, 而 2.7 节中的方案需要执行高达 2^{64} 次 $\binom{2}{1}$ -OT, 因此本节的方案具有明显的优势。在本文的实验中, 对于一个大小为 $H_i \times W_i$ 的输入通道, 利用 EMP 库里的 INKP 协议, 可以将 $H_i W_i$ 次 $\binom{2}{1}$ -OT, 简化为 K 次 $\binom{2}{1}$ -OT, 其中 K 为较小常数。因此在 64 位机上, 对一个输入通道执行一次 YMP 协议的复杂度为 $632K$ 次 $\binom{2}{1}$ -OT。

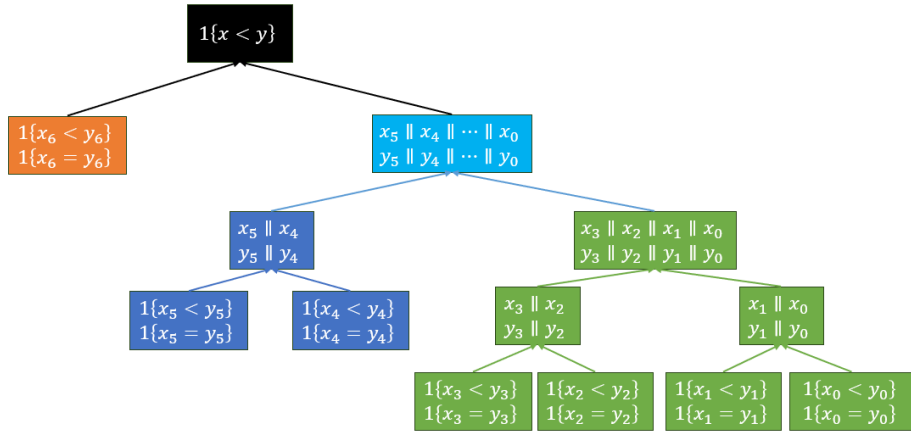


图 3.2 $q=7$ 的 YMP 协议流程图

最后考虑 $m \nmid L$ 且 $2^\alpha < q < 2^{\alpha+1}, \alpha \in \mathbb{N}$ 的情况。当 $m \nmid L$ 时, 可以在 x 和 y 后补 $m - L \bmod m$ 个 0 得到 L' , 使得 $m \mid L'$ 。当 $2^\alpha < q < 2^{\alpha+1}, \alpha \in \mathbb{N}$ 时, 考虑 q 的二进制表示 $q = 2^{i_k} + 2^{i_{k-1}} + \dots + 2^{i_1} (i_k > i_{k-1} > \dots > i_1)$, 分别对 $2^{i_k}, 2^{i_{k-1}}, \dots, 2^{i_1}$ 个底层结点自下而上地进行递归合并, 得到 k 个满二叉树的根结点。然后对该 k 个结点重复上述操作, 直到所有结点合并成一棵二叉树, 最终递归合并次数仍为 $q-1$, 因而复杂度仍保持不变, 图 3.2 展示了 $q=7$ 的分治结构图。

第二节 计算 ReLU 函数的相关协议

本节将利用上一节的 YMP 协议实现计算 DReLU 函数的布尔秘密共享, 并进一步实现计算 ReLU 函数的算术秘密共享。注意到 ReLU 函数与 DReLU 函数有如下关系^[19]

$$\text{ReLU}(x) = x \cdot \text{DReLU}(x), x \in \llbracket n \rrbracket, \quad (3.3)$$

因此当参与方 P_0 和 P_1 拥有 $\text{DReLU}(x)$ 的布尔秘密共享 $c = \langle c \rangle_0^B + \langle c \rangle_1^B$ ，以及 x 的算术秘密共享 $x = \langle x \rangle_0^n + \langle x \rangle_1^n$ 时，可利用如下多路转换器（Multiplexer, Mux），实现计算 $\text{ReLU}(x)$ 的布尔秘密共享 $z = \langle z \rangle_0^n + \langle z \rangle_1^n$ ：

1. $P_i (i = 0, 1)$ 随机选取 $r_i \in \llbracket n \rrbracket$ ；
2. P_0 按如下方式计算有序对 (s_0, s_1) ，并将其作为发送方 $\binom{2}{1}$ -OT 的输入

$$(s_0, s_1) = \begin{cases} (-r_0, -r_0 + \langle x \rangle_0^n), \langle c \rangle_0^B = 0, \\ (-r_0 + \langle x \rangle_0^n, -r_0), \langle c \rangle_0^B = 1. \end{cases} \quad (3.4)$$

3. P_1 将 $\langle c \rangle_1^B$ 作为接收方 $\binom{2}{1}$ -OT 的输入，接受信息 s ；
4. 相对的， P_1 按如下方式计算有序对 (t_0, t_1) ，并将其作为发送方 $\binom{2}{1}$ -OT 的输入

$$(t_0, t_1) = \begin{cases} (-r_1, -r_1 + \langle x \rangle_1^n), \langle c \rangle_1^B = 0, \\ (-r_1 + \langle x \rangle_1^n, -r_1), \langle c \rangle_1^B = 1. \end{cases} \quad (3.5)$$

5. P_0 将 $\langle c \rangle_0^B$ 作为接收方 $\binom{2}{1}$ -OT 的输入，接受信息 t ；
6. P_0 计算 $\langle z \rangle_0^n = r_0 + s$ 作为协议输出， P_1 计算 $\langle z \rangle_1^n = r_1 + t$ 作为协议输出。

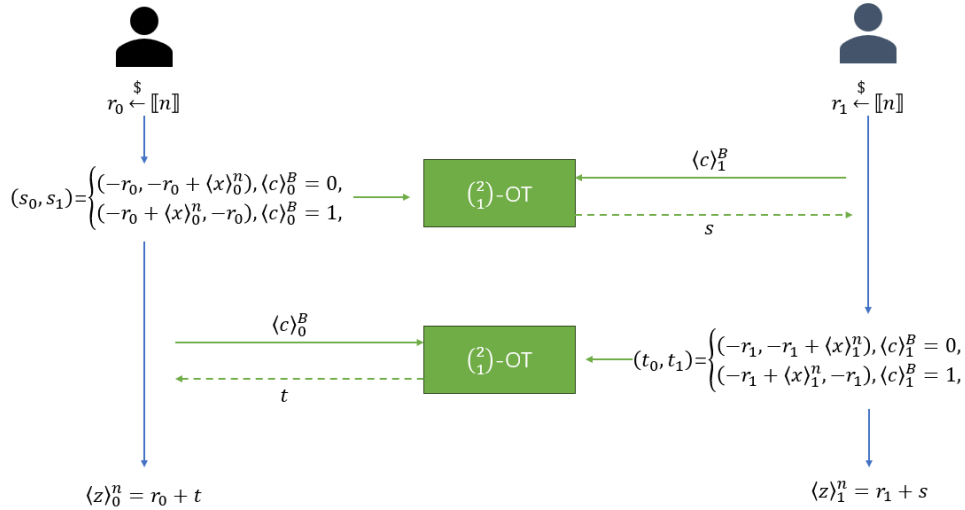


图 3.3 Multiplexer 协议流程图

图 3.3 展示了该协议流程。下面只需实现计算 $\text{DReLU}(x)$ 的布尔秘密共享即

可，首先注意到在环 $\llbracket n \rrbracket$ 上的 $\text{DReLU}(x)$ 函数可表示为^[20]：

$$\text{DReLU}(x) = \begin{cases} 1, x \in [0, \lceil n/2 \rceil], \\ 0, \text{otherwise.} \end{cases} = \begin{cases} 1, \langle x \rangle_0^n + \langle x \rangle_1^n \in [0, \lceil n/2 \rceil] \cup [n, \lceil 3n/2 \rceil], \\ 0, \text{otherwise.} \end{cases} \quad (3.6)$$

(3.6) 式揭示了 DReLU 的本质即为数值大小的比较，从而可以将计算 $\text{DReLU}(x)$ 的布尔秘密共享，等价于与 $\langle x \rangle_0^n, \langle x \rangle_1^n$ 相关的 YMP。即分别做 $n-1-\langle x \rangle_0^n$ 与 $(n-1)/2+\langle x \rangle_1^n$ 、 $n-1-\langle x \rangle_0^n$ 与 $\langle x \rangle_1^n$ 以及 $n+(n-1)/2-\langle x \rangle_0^n$ 与 $\langle x \rangle_1^n$ 之间的大小比较。注意到当 $\langle x \rangle_0^n > (n-1)/2$ 时，必有 $x > \lceil n/2 \rceil$ ；当 $\langle x \rangle_0^n \leq (n-1)/2$ 时，必有 $x < \lceil 3n/2 \rceil$ ，因此只需进行两次大小比较。该协议流程如图 3.4 所示，具体执行如下协议：

1. P_0 和 P_1 分别将 $3(n-1)/2-\langle x \rangle_0^n$ 和 $(n-1)/2+\langle x \rangle_1^n$ 作为 YMP 协议的输入，执行 YMP 协议分别获得输出 $\langle a \rangle_0^B$ 和 $\langle a \rangle_1^B$ ；
2. P_0 和 P_1 分别将

$$m_0 = \begin{cases} n-1-\langle x \rangle_0^n, \langle x \rangle_0^n \leq (n-1)/2, \\ 2n-1-\langle x \rangle_0^n, \langle x \rangle_0^n > (n-1)/2. \end{cases} \quad (3.7)$$

和 $m_1 = (n-1)/2+\langle x \rangle_1^n$ 作为 YMP 协议的输入，执行 YMP 协议分别获得输出 $\langle b \rangle_0^B$ 和 $\langle b \rangle_1^B$ ；

3. P_0 首先随机选取 $\langle z \rangle_0^B \in \mathbb{Z}_2$ 。然后对于 $j = j_0 || j_1 = 0, 1, 2, 3$ ，计算 $s_j = \langle a \rangle_0^B \oplus j_0$ 表示步骤 1 中关于 $\mathbf{1}\{x > n\}$ 的计算结果， $t_j = 1 \oplus \langle b \rangle_0^B \oplus j_1$ 表示步骤 2 中关于 $\mathbf{1}\{x < \lceil n/2 \rceil\}$ 或 $\mathbf{1}\{x < \lceil 3n/2 \rceil\}$ 的计算结果。最后设置

$$z_j = \langle z \rangle_0^B \oplus \begin{cases} s_j \wedge t_j, \langle x \rangle_0^n > (n-1)/2, \\ (s_j \wedge (1 \oplus t_j)) \oplus t_j, \langle x \rangle_0^n \leq (n-1)/2. \end{cases} \quad (3.8)$$

4. P_0 将 $\{z_j\}_j$ 作为发送方 $\binom{4}{1}$ -OT 的输入， P_1 将 $\langle a \rangle_1^B || \langle b \rangle_1^B$ 作为接收方 $\binom{4}{1}$ -OT 的输入，接受信息 $\langle z \rangle_1^B$ 。
5. P_0 和 P_1 分别将 $\langle z \rangle_0^B$ 和 $\langle z \rangle_1^B$ 作为协议输出。

在此以 $\langle x \rangle_0^n = 1, \langle x \rangle_1^n = 2, n = 5$ 的情况为例进行说明，此时 $x = 3$ 在定点数表示下为 -2 ，因此 $\langle z \rangle_0^B + \langle z \rangle_1^B$ 应该为 0。注意到 $3(n-1)/2-\langle x \rangle_0^n = 5$ 以及 $(n-1)/2+\langle x \rangle_1^n = 4$ ，从而不妨设 YMP 协议的结果为 $\langle a \rangle_0^B = 0, \langle a \rangle_1^B = 0$ 。另一边，

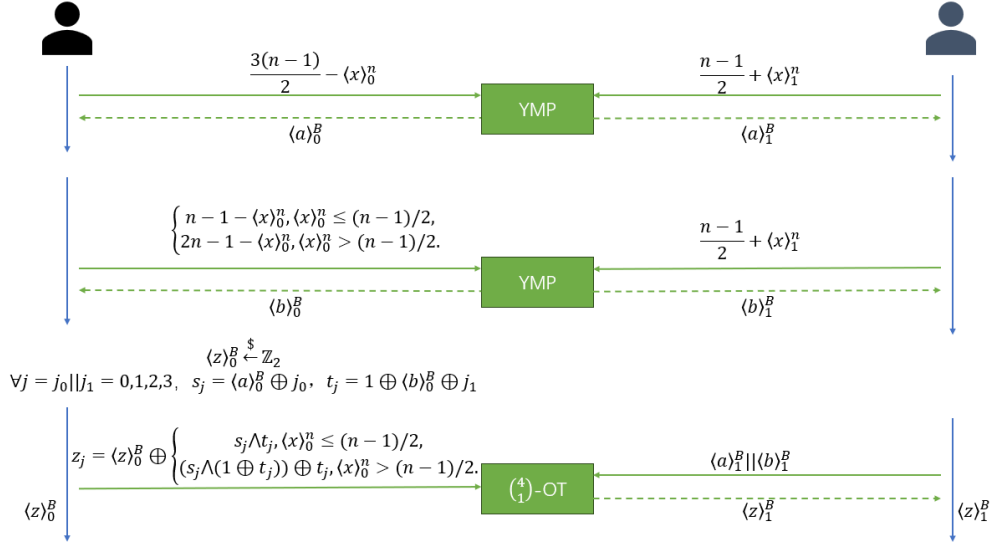


图 3.4 DReLU 协议流程图

$m_0 = 3$ 以及 $m_1 = 4$ ，从而不妨设 YMP 协议的结果为 $\langle b \rangle_0^B = 0, \langle b \rangle_1^B = 1$ 。接着设 P_0 的随机值 $\langle z \rangle_0^B = 1$ ，则有

$$s_0 = 0, s_1 = 0, s_2 = 1, s_3 = 1;$$

$$t_0 = 1, t_1 = 0, t_2 = 1, t_3 = 0,$$

进一步有

$$z_0 = 0, z_1 = 1, z_2 = 0, z_3 = 0.$$

另一边， P_1 的 $\langle a \rangle_1^B || \langle b \rangle_1^B = (01)_2 = 1$ ，从而通过 (4_1) -OT 得到 $\langle z \rangle_1^B = 1$ ，满足 $\langle z \rangle_0^B + \langle z \rangle_1^B = 0$ 。

下面进行复杂度分析，Multiplexer 协议一共调用了两次 (2_1) -OT，时间开销较低。而 DReLU 协议一共调用了两次 YMP 和一次 (4_1) -OT，其时间开销约为两次 YMP 的开销，在可接受范围之内。

第三节 线性层与非线性层的联合优化

3.3.1 $f_r(f_c(x))$ 的展开式

基于共享乘法协议的实现思想：先利用代数变形将乘法函数展开，然后把展示式的各项通过本地计算和通信交互分配给参与双方，最终实现计算乘法函

数的算术秘密共享。在本节中同样利用该思想，实现计算卷积函数与 ReLU 函数的复合函数 $f_c(f_r(x))$ 的算术秘密共享，本小节首先介绍 $f_c(f_r(x))$ 的展开方案。首先注意到有如下等式

$$a \oplus b = a + b - 2ab, \quad (3.9)$$

其中 $a, b \in \mathbb{Z}_2$ ，该式将布尔类型的与运算展开为算术类型运算。下面基于 (3.9) 式，将 $f_c(f_r(x))$ 按如下方式展开

$$\begin{aligned} f_c(f_r(x)) &= k * [x \boxtimes f'_r(x)] \\ &= k * \{(x_0 + x_1) \boxtimes [g_0(x) \oplus g_1(x)]\} \\ &= k * \{(x_0 + x_1) \boxtimes [g_0(x) + g_1(x) - 2 \boxtimes g_0(x) \boxtimes g_1(x)]\} \\ &= k * \{x_0 \boxtimes g_0(x) + g_1(x) \boxtimes x_0 \boxtimes [1 - 2 \boxtimes g_0(x)] \\ &\quad + x_1 \boxtimes g_1(x) + g_0(x) \boxtimes x_1 \boxtimes [1 - 2 \boxtimes g_1(x)]\} \\ &= k * h_1 + k * [g_1(x) \boxtimes h_2 + g_0(x) \boxtimes h_3 + h_4] \\ &= k * h_1 + k * h_5, \end{aligned} \quad (3.10)$$

这里

$$\begin{cases} h_1 = x_0 \boxtimes g_0(x), \\ h_2 = x_0 \boxtimes [1 - 2 \boxtimes g_0(x)], \\ h_3 = x_1 \boxtimes [1 - 2 \boxtimes g_1(x)], \\ h_4 = x_1 \boxtimes g_1(x), \\ h_5 = g_1(x) \boxtimes h_2 + g_0(x) \boxtimes h_3 + h_4, \end{cases} \quad (3.11)$$

其中 x_0, x_1 是 $x \in \llbracket p \rrbracket^{C_i \times H_i \times W_i}$ 的算术秘密共享， $g_0(x), g_1(x)$ 是 DReLU 函数 $f'_r(x) \in \mathbb{Z}_2^{C_i \times H_i \times W_i}$ 的布尔秘密共享， $k \in \llbracket p \rrbracket^{C_o \times C_i \times f_h \times f_h}$ 为卷积核，素数 p 为明文模数。

3.3.2 无旋卷积协议

在一般的密文卷积运算中，利用 BatchEncode 技术对每个 $H_i \times W_i$ 的二维通道进行编码，此时会引入大量的旋转操作。旋转操作包含多次数论变换 (Number-Theoretic Transform, NTT) 和逆数论变换 (Inverse Number-Theoretic Transform, INTT)，因此旋转操作的复杂度远高于加法操作和乘法操作的复杂

度^[21]，进而导致卷积运算的时间开销骤然上升。本小节将基于简单的矩阵变换，将卷积运算等价为矩阵乘法运算，并实现消除旋转操作的卷积运算。假设参与方 P_0 具有卷积核 $k \in \llbracket p \rrbracket^{C_o \times C_i \times f_h \times f_h}$ ，参与方 P_1 具有输入 $x \in \llbracket p \rrbracket^{C_i \times H_i \times W_i}$ ，且双方均知道步长 s 。 P_0 和 P_1 执行如下协议，该协议流程如图 3.5 所示，使得在 P_0 不泄露 k 且 P_1 不泄露 x 的情况下， P_0 和 P_1 获得 $k * x \in \llbracket p \rrbracket^{C_o \times H_o \times W_o}$ 的算术秘密共享：

1. 编码与加密： P_0 与 P_1 分别按 (3.12) 式和 (3.13) 式所示的方式调整 k 和 x 的结构，得到矩阵 $\hat{k} \in \llbracket p \rrbracket^{C_o \times C_i(f_h)^2}$ 和 $\hat{x} \in \llbracket p \rrbracket^{C_i(f_h)^2 \times H_o W_o}$ ，使得矩阵乘法 $\hat{k} \cdot \hat{x}$ 的每个行向量的元素，与卷积 $k * x$ 的每个通道的元素对应相等。

$$\begin{aligned} \forall i \in \llbracket C_i(f_h)^2 \rrbracket \wedge \forall j \in \llbracket H_o W_o \rrbracket, \hat{x}_{i,j} = x_{c,h,w}, \text{ where } c = \lfloor \frac{i}{(f_h)^2} \rfloor, \\ c' = i \bmod (f_h)^2, h = \lfloor \frac{j}{W_o} \rfloor \cdot s + \lfloor \frac{c'}{f_h} \rfloor, w = (j \bmod W_o) \cdot s + (c' \bmod f_h). \end{aligned} \quad (3.12)$$

$$\begin{aligned} \forall i \in \llbracket C_o \rrbracket \wedge \forall j \in \llbracket C_i(f_h)^2 \rrbracket, \hat{k}_{i,j} = k_{u,v,h,w}, \text{ where } u = i, v = \lfloor \frac{j}{(f_h)^2} \rfloor, \\ h = \lfloor \frac{j}{f_h} \rfloor \bmod f_h, w = j \bmod f_h. \end{aligned} \quad (3.13)$$

设明文向量长度为 N ， P_1 将 $\hat{x}$ 的每 $C_n = \lfloor N/H_o W_o \rfloor$ 个行向量打包成一个明文 $\hat{x}_j (j = 1, 2, \dots, R_n)$ ，并加密得到 R_n 个密文 $[\hat{x}_j]$ ，其中 $R_n = \lceil C_i(f_h)^2/C_n \rceil$ 。对于 $\hat{k}$ 的每个行向量 $\hat{k}_i (i = 1, 2, \dots, C_o)$ ， P_0 将 $\hat{k}_i$ 的每 C_n 个元素，扩展成一个长度为 N 的明文向量，其中每个元素按序占据 $H_o W_o$ 个相邻位置，一共得到 $C_o R_n$ 个明文 $\hat{k}_{i,j}$ 。

2. 乘积与求和： P_1 将 $[\hat{x}_j]$ 发送给 P_0 。对于每个 i ， P_0 计算 $\sum_j \hat{k}_{i,j} \boxtimes [\hat{x}_j]$ ，并引入混淆项 r_i ，将 $\sum_j \hat{k}_{i,j} \boxtimes [\hat{x}_j] - r_i$ 发送给 P_1 。
3. 解密再求和：对于每个 i ， P_1 将 $\sum_j \hat{k}_{i,j} \boxtimes [\hat{x}_j] - r_i$ 解密得到向量 $\widehat{res}_i^1 = \sum_j \hat{k}_{i,j} \boxtimes \hat{x}_j - r_i$ ，然后将 $\widehat{res}_i^1$ 解包成 C_n 个长度为 $H_o W_o$ 的向量，并求和相加得到 res_i^1 。对于每个 i ， P_0 将 r_i 解包成 C_n 个长度为 $H_o W_o$ 的向量，并求和得到 res_i^0 。此时每个 $res_i^0 + res_i^1$ 即为 $k * x$ 的第 i 个二维通道。

上述协议一共执行了 R_n 次加密运算、 C_o 次解密运算、 $C_o \cdot R_n$ 次密文与明文的乘法和 $C_o \cdot R_n$ 次密文与密文的加法。与多项式逼近方案相比，仅多出了少量的加密和解密运算，并减少了因测试样例过少而造成的外部大碎片，但也造成了因用多个明文向量编码一个二维通道的内部小碎片。

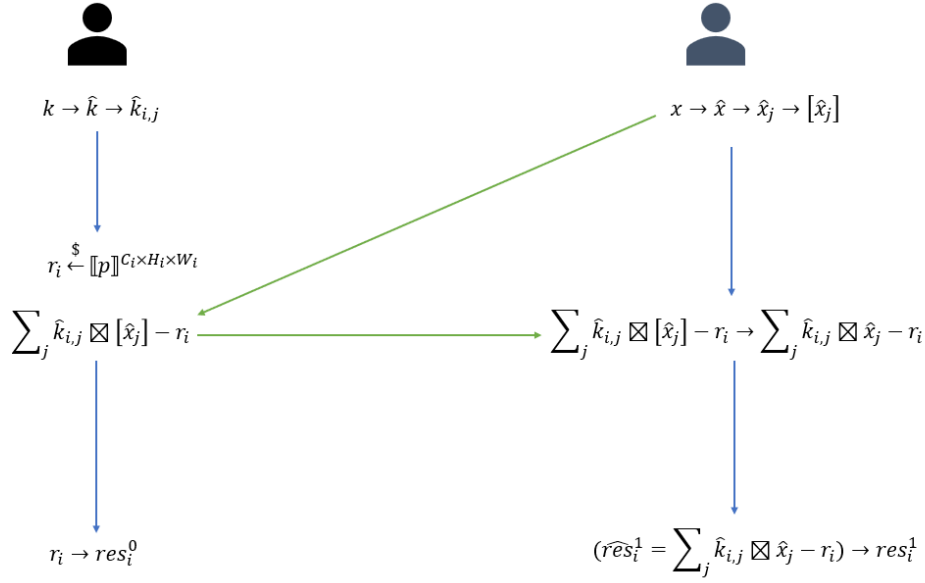
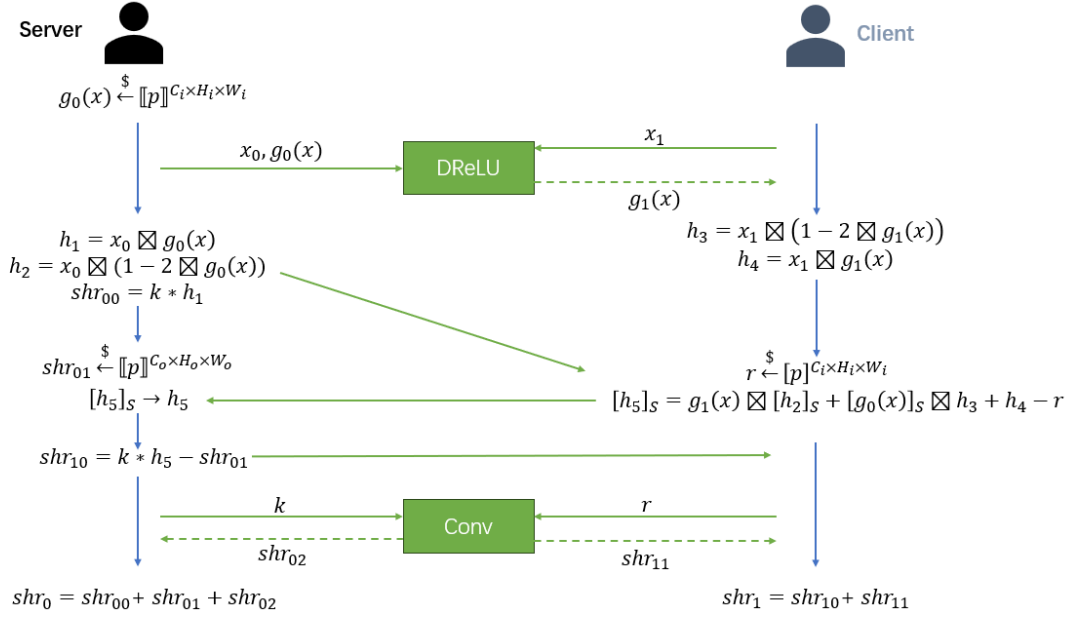


图 3.5 无旋卷积协议流程图

3.3.3 $f_c(f_r(x))$ 协议

基于 3.3.1 节和 3.3.2 节的方案，下面通过本地计算和通信交互，实现将 $k * h_1$ 和 $k * h_5$ 分配给服务器和用户作为 $f_c(f_r(x))$ 的算术秘密共享的协议：

1. 假设服务器作为参与方 P_0 持有秘密份额 x_0 和卷积核 k ，并预生成秘密份额 $g_0(x) \xleftarrow{\$} \mathbb{Z}_2^{C_i \times H_i \times W_i}$ ，用户作为参与方 P_1 持有秘密份额 x_1 ，并与服务器执行 DReLU 协议获得秘密份额 $g_1(x)$ 。
2. 此时服务器在本地计算 h_1 和 h_2 ，用户在本地计算 h_3 和 h_4 ，进一步服务器在本地计算 $shr_{00} = k * h_1$ ；
3. 然后服务器为了让用户获得 h_5 ，并避免 h_2 和 $g_0(x)$ 的泄露，将利用 HE 加密的 $[h_2]_S$ 和 $[g_0(x)]_S$ ，一共 $2\lceil C_i / \lfloor N / (H_i W_i) \rfloor \rceil$ 发送给用户。
4. 接着用户为了获取 $k * h_5$ 的结果，并避免 $g_1(x), h_3, h_4$ 的泄露，计算 h_5 的密文 $[h_5]_S = g_1(x) \boxtimes [h_2]_S + [g_0(x)]_S \boxtimes h_3 + h_4$ ，并引入混淆项 r ，将 $[h_5]_S - r$ 返回给服务器。
5. 服务器解密接收消息得到 $h'_5 = h_5 - r$ ，并计算 $k * h'_5$ 。服务器为了将 $k * h'_5$ 返回给用户，并避免 k 的泄露，引入混淆项 shr_{01} ，并将 $shr_{10} = k * h'_5 - shr_{01}$ 返回给用户。
6. 服务器和用户利用 3.3.2 节中的无旋卷积协议计算 $k * r$ ，分别获得秘密份额 shr_{02} 和 shr_{11} 。最后服务器和用户分别计算 $shr_0 = shr_{00} + shr_{01} + shr_{02}$ 和 $shr_1 = shr_{10} + shr_{11}$ 作为 $f_c(f_r(x))$ 的算术秘密共享。


 图 3.6 $f_c(f_r(x))$ 协议流程图

该协议流程如图 3.6 所示。下面进行复杂度分析， $f_c(f_r(x))$ 协议一共执行了一次 DReLU 协议、一次无旋卷积协议、 $2\lceil C_i / \lfloor N / (H_i W_i) \rfloor \rceil$ 次加密运算、 $\lceil C_i / \lfloor N / (H_i W_i) \rfloor \rceil$ 次解密运算、 $2\lceil C_i / \lfloor N / (H_i W_i) \rfloor \rceil$ 次密文与明文的乘法、 $2\lceil C_i / \lfloor N / (H_i W_i) \rfloor \rceil$ 次密文与密文的加法和 $\lceil C_i / \lfloor N / (H_i W_i) \rfloor \rceil$ 次密文与明文的加法，其中时间开销主要由一次 DReLU 协议和一次无旋卷积协议占据。

3.3.4 $f_w(f_c(x))$ 的展开式

同样地，按照计算 $f_c(f_r(x))$ 的思路，计算全连接函数与 ReLU 函数的复合函数 $f_w(f_r(x))$ 的算术秘密共享，下面将 $f_w(f_r(x))$ 按如下方式展开：

$$\begin{aligned}
 f_w(f_r(x)) &= w \cdot [x \boxtimes f'_r(x)] \\
 &= w \cdot \{(x_0 + x_1) \boxtimes [g_0(x) \oplus g_1(x)]\} \\
 &= w \cdot \{(x_0 + x_1) \boxtimes [g_0(x) + g_1(x) - 2 \boxtimes g_0(x) \boxtimes g_1(x)]\} \\
 &= w \cdot \{x_0 \boxtimes g_0(x) + g_1(x) \boxtimes x_0 \boxtimes [1 - 2 \boxtimes g_0(x)] \\
 &\quad + x_1 \boxtimes g_1(x) + g_0(x) \boxtimes x_1 \boxtimes [1 - 2 \boxtimes g_1(x)]\} \\
 &= w \cdot h_1 + w \cdot [g_1(x) \boxtimes h_2 + g_0(x) \boxtimes h_3 + h_4] \\
 &= w \cdot h_1 + w \cdot h_5,
 \end{aligned} \tag{3.14}$$

这里

$$\begin{cases} h_1 = x_0 \boxtimes g_0(x), \\ h_2 = x_0 \boxtimes [1 - 2 \boxtimes g_0(x)], \\ h_3 = x_1 \boxtimes [1 - 2 \boxtimes g_1(x)], \\ h_4 = x_1 \boxtimes g_1(x), \\ h_5 = g_1(x) \boxtimes h_2 + g_0(x) \boxtimes h_3 + h_4, \end{cases} \quad (3.15)$$

其中 x_0, x_1 是 $x \in \llbracket p \rrbracket^{N_i}$ 的算术秘密共享, $g_0(x), g_1(x)$ 是 DReLU 函数 $f'_r(x) \in \mathbb{Z}_2^{N_i}$ 的布尔秘密共享, $w \in \llbracket p \rrbracket^{N_o \times N_i}$ 为权值矩阵, 素数 p 为明文模数。

3.3.5 无旋矩阵乘法协议

类似地, 本小节实现消除旋转操作的矩阵乘法运算。假设参与方 P_0 具有权值矩阵 $w \in \llbracket p \rrbracket^{N_o \times N_i}$, 参与方 P_1 具有输入 $x \in \llbracket p \rrbracket^{N_i}$ 。 P_0 和 P_1 执行如下协议, 该协议流程如图 3.7 所示, 使得 P_0 在不泄露 w 的情况下, P_0 和 P_1 获得 $w \cdot x \in \llbracket p \rrbracket^{N_o}$ 的算术秘密共享:

1. 编码与加密: 设明文向量长度为 N , P_1 将 x 的每 $C_n = \lfloor N/N_o \rfloor$ 个元素打包成一个明文 $\hat{x}_j$, 其中每个元素按序占据 N_o 个相邻位置, 最终加密得到 $R_n = \lceil N_i/C_n \rceil$ 个密文 $[\hat{x}_j] (j = 1, 2, \dots, R_n)$ 。 P_0 将 w 的每 C_n 个列向量, 打包成一个长度为 N 的明文向量, 得到 R_n 个明文 $\hat{w}_j$ 。
2. 乘积与求和: P_1 将 $[\hat{x}_j]$ 发送给 P_0 。 P_0 计算 $\sum_j \hat{w}_j \boxtimes [\hat{x}_j]$, 并引入混淆项 r , 将 $\sum_j \hat{w}_j \boxtimes [\hat{x}_j] - r$ 发送给 P_1 。
3. 解密再求和: P_1 将 $\sum_j \hat{w}_j \boxtimes [\hat{x}_j] - r$ 解密得到向量 $\widehat{res}^1 = \sum_j \hat{w}_j \boxtimes \hat{x}_j - r$, 然后将 $\widehat{res}^1$ 解包成 C_n 个长度为 N_o 的向量, 并求和相加得到 res^1 。 P_0 将 r 解包成 C_n 个长度为 N_o 的向量, 并求和相加得到 res^0 。此时 $res^0 + res^1$ 即为 $w \cdot x$ 。

上述协议一共执行了 R_n 次加密运算、一次解密运算、 R_n 次密文与明文的乘法和 R_n 次密文与密文的加法。与多项式逼近方案相比, 同样仅多出了少量的加密和解密运算, 并减少了因测试样例过少而造成的外部大碎片, 但因此造成了因用多个明文向量编码一个二维通道的内部小碎片。

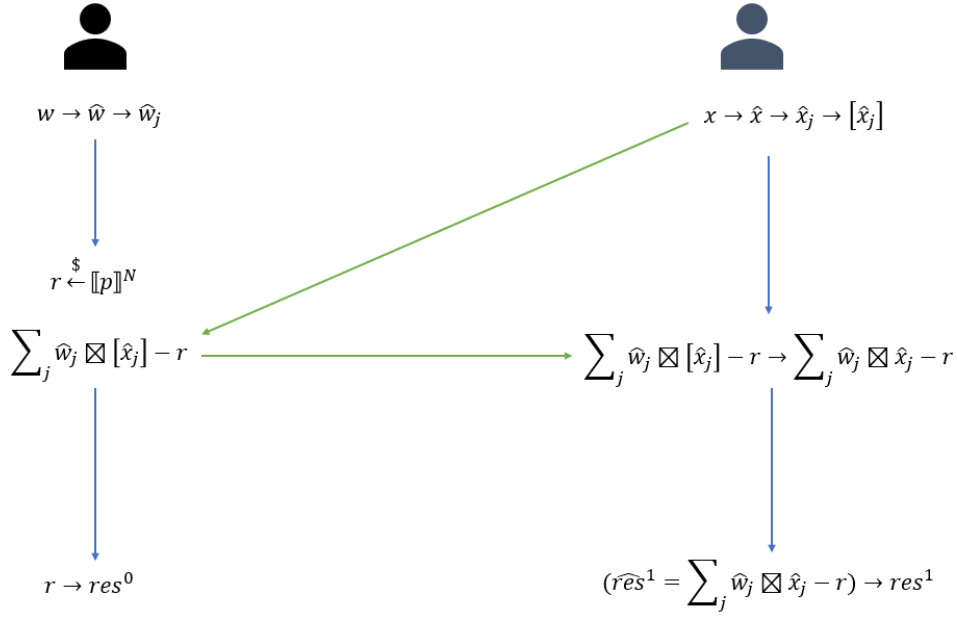
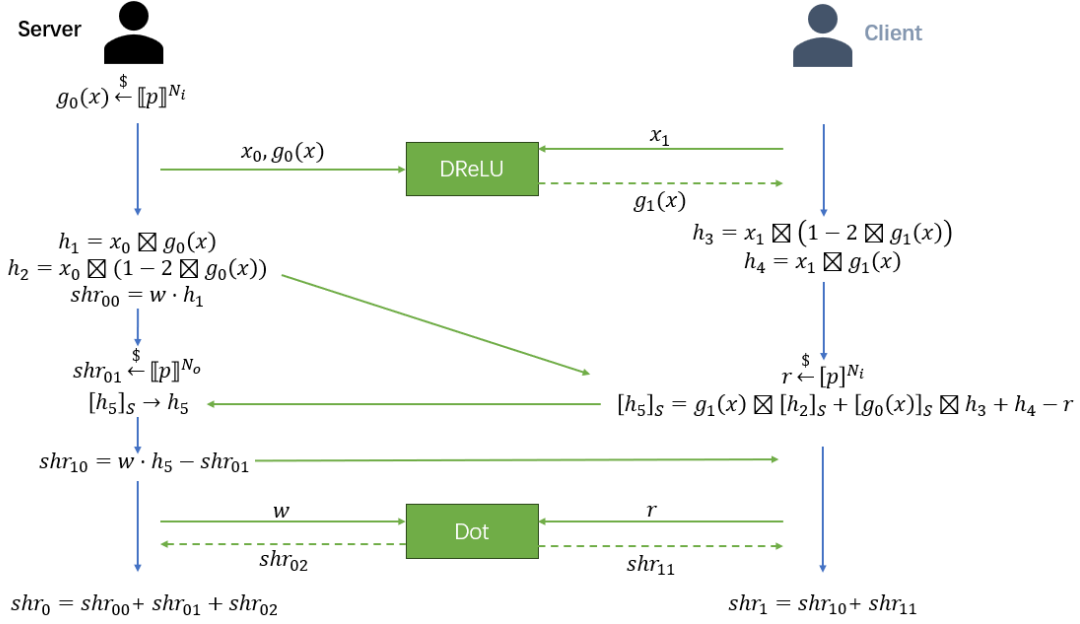


图 3.7 无旋矩阵乘法协议流程图

3.3.6 $f_w(f_r(x))$ 协议

基于 3.3.4 节和 3.3.5 节的方案，下面实现将 $w \cdot h_1$ 和 $w \cdot h_5$ 分别分配给服务器和用户作为 $f_w(f_r(x))$ 的算术秘密共享的协议：

1. 假设服务器作为参与方 P_0 持有秘密份额 x_0 和权值矩阵 w ，并预生成秘密份额 $g_0(x) \xleftarrow{\$} \mathbb{Z}_2^{N_i}$ ，用户作为参与方 P_1 持有秘密份额 x_1 ，并与服务器执行 DReLU 协议获得秘密份额 $g_1(x)$ 。
2. 此时服务器在本地计算 h_1, h_2 ，用户在本地计算 h_3, h_4 ，进一步服务器在本地计算 $shr_{00} = w \cdot h_1$ ；
3. 然后服务器为了让用户获得 h_5 ，并避免 $h_2, g_0(x)$ 的泄露，将利用 HE 技术加密的 $[h_2]_S$ 和 $[g_0(x)]_S$ ，一共 $2 \cdot \lceil N_i/N \rceil$ 发送给用户。
4. 接着用户为了获取 $w \cdot h_5$ 的结果，并避免 $g_1(x), h_3, h_4$ 的泄露，计算 h_5 的密文 $[h_5]_S = g_1(x) \boxtimes [h_2]_S + [g_0(x)]_S \boxtimes h_3 + h_4$ ，并引入混淆项 r ，将 $[h_5]_S - r$ 返回给服务器。
5. 服务器解密接收消息得到 $h'_5 = h_5 - r$ ，并计算 $w \cdot h'_5$ 。服务器为了将 $w \cdot h'_5$ 返回给用户，并避免 w 的泄露，引入混淆项 shr_{01} ，并将 $shr_{10} = w \cdot h'_5 - shr_{01}$ 返回给用户。
6. 服务器和用户利用 3.3.5 节无旋卷积协议计算 $w \cdot r$ ，分别获得秘密份额 shr_{02} 和 shr_{11} 。最后服务器和用户分别计算 $shr_0 = shr_{00} + shr_{01} + shr_{02}$ 和 $shr_1 = shr_{10} + shr_{11}$ 作为 $f_w(f_r(x))$ 的算术秘密共享。


 图 3.8 $f_w(f_r(x))$ 协议流程图

该协议流程如图 3.8 所示，下面进行复杂度分析， $f_w(f_r(x))$ 协议一共执行了一次 DReLU 协议、一次无旋矩阵乘法协议、 $2\lceil N_i/N \rceil$ 次加密运算、 $\lceil N_i/N \rceil$ 次解密运算、 $2\lceil N_i/N \rceil$ 次密文与明文的乘法、 $2\lceil N_i/N \rceil$ 次密文与密文的加法和 $\lceil N_i/N \rceil$ 次密文与明文的加法，其中时间开销主要由一次 DReLU 协议和一次无旋卷积协议占据。

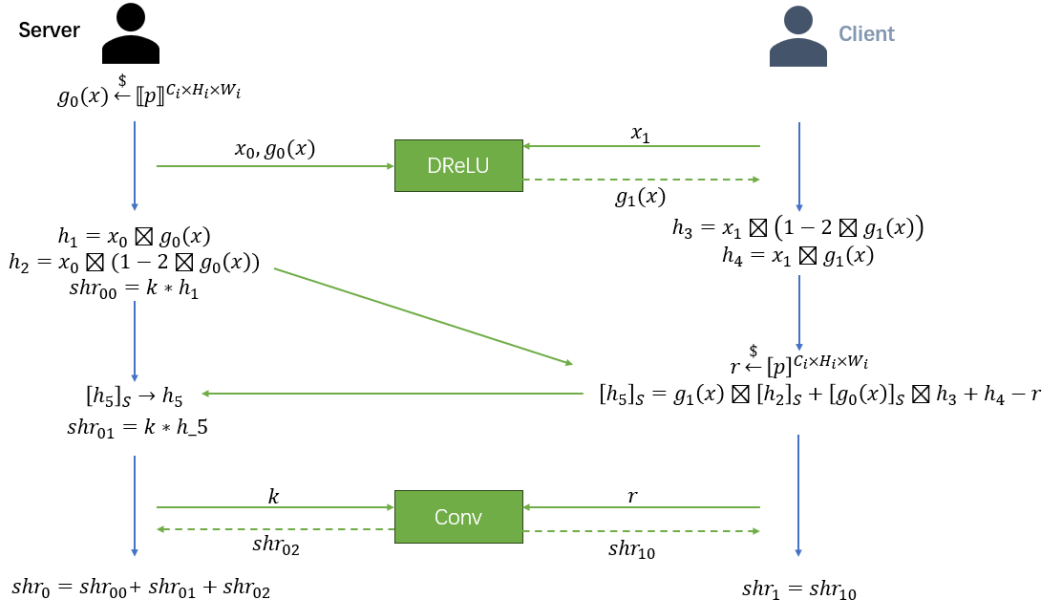
3.3.7 $f_c(f_r(x))$ 和 $f_w(f_r(x))$ 协议的优化

在网络环境中，相对于计算开销，通信开销的时间花费更大。3.3.3 节和 3.3.6 节中的协议中除一次 DReLU 协议和一次无旋卷积协议，还需要执行 3 次通信交互。注意到在第 3 次通信中，服务器仅是将结果 shr_{10} 与用户分享，若不进行分享并不会产生安全隐患。因此服务器可以省去随机选取 shr_{01} 的过程，将 $k \cdot h_5$ 也作为服务器的秘密共享的一部分，从而省去了第 3 次通信。具体的 $f_c(f_r(x))$ 协议流程如图 3.9， $f_w(f_r(x))$ 协议也可以采用同样的方式进行优化。

第四节 共享除法协议

为了实现定点数编码的数据截断，本节将实现算术秘密共享的共享除法协议。首先定义定点数除法和环上除法：

- 定点数除法 $\text{idiv} : \mathbb{Z}_p \times (\mathbb{N} \cap [1, p)) \rightarrow \mathbb{Z}_p$ ：设 $a \in \mathbb{Z}_p$ 为被除数， $d \in \{1, 2, \dots, p-1\}$


 图 3.9 优化版 $f_c(f_r(x))$ 协议流程图

1} 为除数，则商为 a/d 向 $-\infty$ 舍入的值。

- 环上除法 $\mathbf{rdiv} : \llbracket p \rrbracket \times (\mathbb{N} \cap [1, p)) \rightarrow \llbracket p \rrbracket$: 设 $a \in \llbracket p \rrbracket$ 为被除数, $d \in \{1, 2, \dots, p-1\}$ 为除数，则商定义如下：

$$\mathbf{rdiv}(a, d) \triangleq \mathbf{idiv}(a - 1\{a \geq \lceil p/2 \rceil\} \cdot p, d) \bmod p. \quad (3.16)$$

对于秘密 $x \in \llbracket p \rrbracket$ ，假设参与方 P_0 持有秘密份额 x_0 ，参与方 P_1 持有秘密份额 x_1 。当需要执行运算 $\mathbf{rdiv}(x, d)$ 时，其中 $d \in \mathbb{N} \cap [1, p)$ ，一般方案^[22]采用直接用运算 $\mathbf{rdiv}(x_0, d)$ 和 $\mathbf{rdiv}(x_1, d)$ 代替运算 $\mathbf{rdiv}(x, d)$ 。当 x_0 和 x_1 较小时，该方案具有正确性；但当 x_0 或 x_1 较大时，该方案可能出现错误。例如，当 $p = 11, x_0 = 3, x_1 = 9, d = 3$ 时，此时 $x = (3 + 9) \bmod 11 = 1$ ，进而 $\mathbf{rdiv}(x, d) = 0$ ，而 $(\mathbf{rdiv}(x_0, d) + \mathbf{rdiv}(x_1, d)) \bmod p = 4$ ，与正确值有巨大的误差；当 $p = 11, x_0 = 2, x_1 = 4, d = 2$ 时，此时 $x = (2 + 4) \bmod 11 = 6$ ，进而 $\mathbf{rdiv}(x, d) = 9$ ，而 $((\mathbf{rdiv}(x_0, d) + \mathbf{rdiv}(x_1, d)) \bmod p = 3$ ，与正确值同样有巨大的误差。产生错误的原因有如下两点：

1. 当 $x_0 > \lfloor p/2 \rfloor$ 或 $x_1 > \lfloor p/2 \rfloor$ 时， x_0, x_1 的定点数表示为 $x_0 - p$ 和 $x_1 - p$ ，因此不能直接用 $\mathbf{idiv}(x_0, d)$ 和 $\mathbf{idiv}(x_1, d)$ 代替 $\mathbf{idiv}((x_0 - p), d)$ 和 $\mathbf{idiv}((x_1 - p), d)$ 。
2. 当 $x > \lfloor p/2 \rfloor$ 时，其定点数表示为 $p - x$ ，而当 $x_0 \leq \lfloor p/2 \rfloor$ 且 $x_1 \leq \lfloor p/2 \rfloor$ 时，

不能直接用 $\text{idiv}(x_0, d) + \text{idiv}(x_1, d) = \lfloor x_0/d \rfloor + \lfloor x_1/d \rfloor$ 代替 $\text{idiv}(p-x, d)$ 。因此需要结合 $\mathbf{1}\{x < \lfloor p/2 \rfloor\}$ 、 $\mathbf{1}\{x_0 < \lfloor p/2 \rfloor\}$ 和 $\mathbf{1}\{x_1 < \lfloor p/2 \rfloor\}$ 的结果^[23]，实现正确的算术秘密共享的除法协议。注意到有如下代数变形^[24]：

设 $a \in \llbracket p \rrbracket$ ，其秘密分享为 $a_0, a_1 \in \llbracket p \rrbracket$ ，公共除数 $d \in \mathbb{N} \cap [1, p)$ ，定点数边界 $p' = \lfloor p/2 \rfloor$ ，且 $p = p^1 \cdot d + p^0$ ， $a_0 = a_0^1 \cdot d + a_0^0$ ， $a_1 = a_1^1 \cdot d + a_1^0$ ，其中 $p^1, a_0^1, a_1^1 \in \llbracket p \rrbracket$ 且 $p^0, a_0^0, a_1^0 \in \llbracket d \rrbracket$ 。则有

$$\text{rdiv}(a, d) = (\text{rdiv}(a_0, d) + \text{rdiv}(a_1, d) + (\text{corr} \cdot p^1 + 1 - C - B)) \bmod p \quad (3.17)$$

其中

$$\text{corr} = \begin{cases} -1, (a \geq p') \wedge (a_0 < p') \wedge (a_1 < p'), \\ 1, (a < p') \wedge (a_0 \geq p') \wedge (a_1 \geq p'), \\ 0, \text{otherwise}, \end{cases} \quad (3.18)$$

$$A = a_0^0 + a_1^0 - (\mathbf{1}\{a_0 \geq p'\} + \mathbf{1}\{a_1 \geq p'\} - \text{corr}) \cdot p^0, \quad (3.19)$$

$$B = \text{idiv}(a_0^0 - \mathbf{1}\{a_0 \geq p'\} \cdot p^0, d) + \text{idiv}(a_1^0 - \mathbf{1}\{a_1 \geq p'\} \cdot p^0, d), \quad (3.20)$$

$$C = \mathbf{1}\{A < d\} + \mathbf{1}\{A < 0\} + \mathbf{1}\{A < -d\}. \quad (3.21)$$

上式中 $\text{rdiv}(a_0, d)$ 和 $\text{rdiv}(a_1, d)$ 、 $\mathbf{1}\{a_0 \geq p'\}$ 和 $\mathbf{1}\{a_0 < p'\}$ 、 $\mathbf{1}\{a_1 \geq p'\}$ 和 $\mathbf{1}\{a_1 < p'\}$ 以及 $\text{idiv}(a_0^0 - \mathbf{1}\{a_0 \geq p'\} \cdot p^0, d)$ 和 $\text{idiv}(a_1^0 - \mathbf{1}\{a_1 \geq p'\} \cdot p^0, d)$ 均可在本地计算，因此 B 的算术秘密共享可以在本地计算。而 $\mathbf{1}\{a \geq p'\}$ 和 $\mathbf{1}\{a < p'\}$ 的布尔秘密共享可以通过一次 DReLU 计算，因此可以利用 $\binom{4}{1}$ -OT 计算 corr 的布尔秘密共享，进一步计算 A 的算术秘密共享。接着再利用三次 DReLU 计算 C 的算术秘密共享。最后利用以上结果，即可计算出 $\text{rdiv}(a, d)$ 的算术秘密共享。该除法协议流程由图 3.10 所示，需要执行四次 DReLU 协议和一次 $\binom{4}{1}$ -OT 协议，是本章中计算开销和通信开销最大的协议。

第五节 $\binom{n}{1}$ -OT 协议的优化

2.5.2 节中介绍的 $\binom{n}{1}$ -OT 协议需要执行 n 次 $\binom{2}{1}$ -OT，本文进行二进制优化，实现了复杂度为 $O(\log n)$ 次 $\binom{2}{1}$ -OT 的 $\binom{n}{1}$ -OT 协议。假设 S 持有 n 个信息 $m_0, m_1, \dots, m_{n-1} \in \llbracket p \rrbracket$ ， R 想要获取其中下标为 $c \in \llbracket n \rrbracket$ 的信息，其中 n 的比特串长度为 l ， c 的比特串表示为 $c_{l-1} || c_{l-2} || \dots || c_0$ 。在二分相关的算法中，每次比较

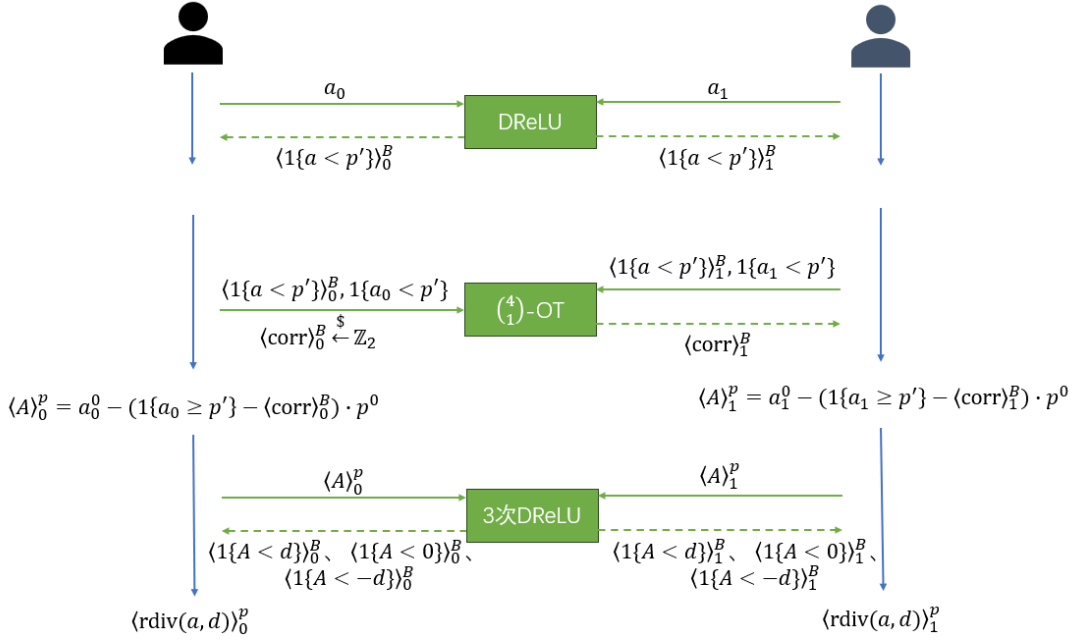
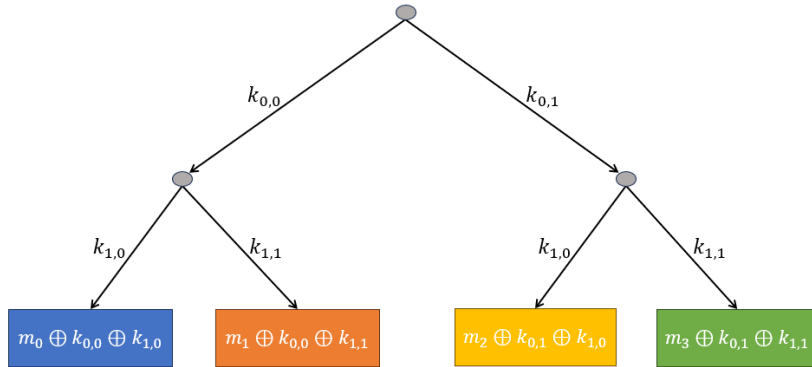


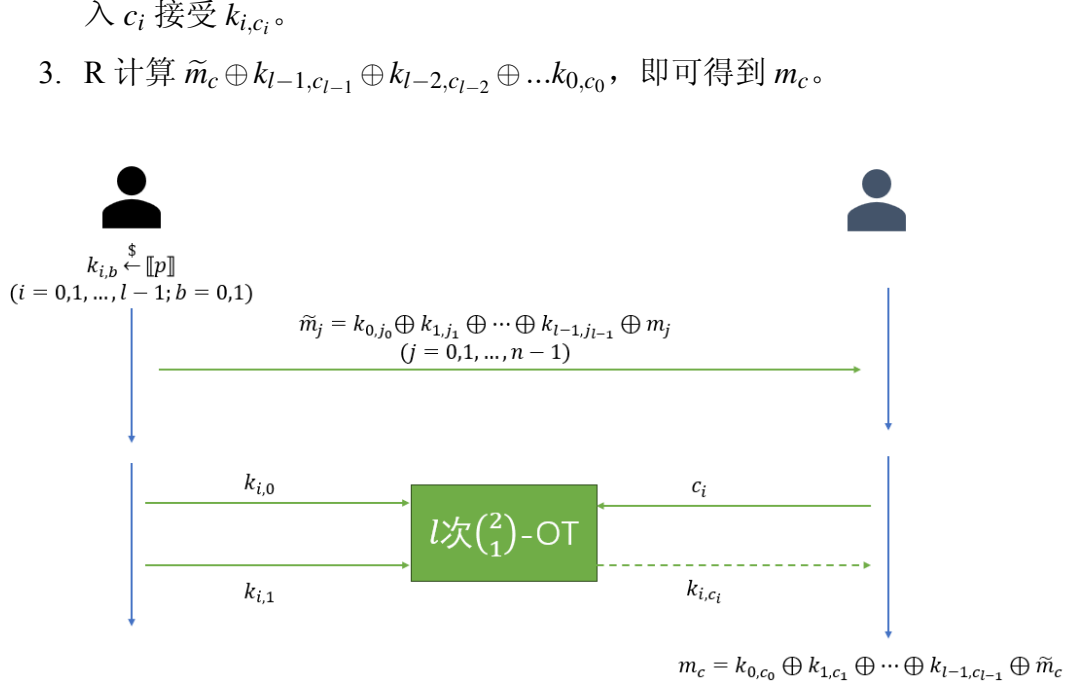
图 3.10 除法协议流程图

都会将范围缩小一半，最终在 $O(\log n)$ 次比较后即可得到结果。在本文中，注意到每次 $\binom{2}{1}$ -OT 可以确定 c 的一个比特位，因此同样在 $O(\log n)$ 次 $\binom{2}{1}$ -OT 后，接收方有办法获得消息 m_c 。下面基于二分算法的二叉树结构设计如下协议：

1. S 随机选取数组 $k_{i,b} \xleftarrow{\$} \llbracket p \rrbracket$ ，其中 $i = 0, 1, \dots, l-1$ 和 $b = 0, 1$ 。然后对于每个 $j = j_{l-1} || j_{l-2} || \dots || j_0 \in \llbracket n \rrbracket$ ，计算 $\tilde{m}_j = k_{l-1, j_{l-1}} \oplus k_{l-2, j_{l-2}} \oplus \dots \oplus k_{0, j_0} \oplus m_j$ 。最后将所有 $\tilde{m}_j$ 发送给 R，图 3.11 展示了该过程的抽象结构。


 图 3.11 $n = 4$ 时优化版 $\binom{2}{1}$ -OT 的 $\tilde{m}_i$ 构造示意图

2. S 和 R 进行 l 次 $\binom{2}{1}$ -OT 协议。在第 i 次协议中，S 输入 $k_{i,0}$ 和 $k_{i,1}$ ，R 输


 图 3.12 优化版 $\binom{n}{1}$ -OT 协议流程图

该协议流程如图 3.12 所示, 相比于 2.5.2 节的方案, 本方案需要多维护一次性密钥 $k_{i,b}$ 和加密消息 $\tilde{m}_i$, 其中 $k_{i,b}$ 包含 $2\log n$ 个整型变量, 比 2.5.1 节中 $O(n)$ 个一次性密钥 k_i 减少了一个数量级, $\tilde{m}_i$ 包含 n 个整型变量, 因而空间复杂度仍为 $O(n)$ 。而在时间复杂度上, 该方案虽需要多进行一次数据传输, 以将密文 $\tilde{m}_i$ 发送给接收方, 但只需要执行 $O(\log n)$ 次 $\binom{2}{1}$ -OT, 比 2.5.2 节中 $O(n)$ 次 $\binom{2}{1}$ -OT 减少了一个数量级, 因此在 n 较大时, 本方案的时间开销更具优势。

第四章 实验与分析

第一节 实验环境

本实验使用虚拟机 VMware 搭建 Ubuntu 系统，并使用代码编辑器 Visual Studio Code，搭建 C++ 编译环境，配置了 SEAL 同态加密库和 EMP 安全多方计算库的运行环境。本实验的数据集为 MNIST，其中包含 10000 张手写数字图片以及图片标签，单张图片的大小为 28×28 ，每个像素点的值用 0/1 表示。本实验采用 socket 进行网络编程，模拟服务器进程和用户进程的通信。具体配置如表 4.1 所示。

表 4.1 环境配置表

VMware	VMware 17.0.1
操作系统	Ubuntu 22.04.5
RAM 容量	4GB
CPU 型号	Intel(R) Core(TM) i7-10875H CPU @ 2.30GHz 2.30 GHz
CPU 内核数	8
代码编辑器	Visual Studio Code 1.99.2
编译器	g++ (Ubuntu 9.5.0-1ubuntu1~22.04) 9.5.0
SEAL	SEAL 4.1.2
EMP	emp-tool 0.2.5-7 和 emp-ot 0.2.4-2

第二节 明文卷积神经网络

本文构建了表 4.2 所示的明文 CNN，一共包含两个卷积层、两个全连接层、两个 ReLU 层以及一个 softmax 层。在上述明文网络的基础上，本文进行了 10 epochs 训练，每轮训练包含训练集里 60000 张图片，得到了用于推理的模型参数。最后使用该模型参数对测试集里的 10000 张图片进行推理测试，成功达到了 97.40% 的准确率。

表 4.2 明文卷积神经网络

	输入大小	卷积核/权值矩阵大小	输出大小	步长大小	填充大小
卷积层 1	28×28	$5 \times 1 \times 5 \times 5$	$5 \times 13 \times 13$	2	1
ReLU 层 1	$5 \times 13 \times 13$		$5 \times 13 \times 13$		
卷积层 2	$5 \times 13 \times 13$	$50 \times 5 \times 5 \times 5$	$50 \times 5 \times 5$	2	
全连接层 1	1250×1	100×1250	100×1		
ReLU 层 2	100×1		100×1		
全连接层 2	100×1	10×100	10×1		
softmax 层	10×1		10×1		

第三节 密文卷积神经网络

4.3.1 加密参数的设置

本文涉及的加密参数主要有明文多项式度数 N 、明文模数 p 和密文模数 q 。各个参数与缩放因子 $scale$ 之间的关系如下：

1. 为了支持 BFV 的批量编码， N 和 p 需要满足 $p = 2KN + 1$ ，其中 $K \in \mathbb{N}$ 。因此 p 的大小与 N 的大小正相关，即当 N 较大时， p 也需要选择较大的素数。
2. 在本文涉及的密文运算中，消耗噪声预算最多的操作为明文与密文的乘法，其消耗的噪声预算与明文的大小线性相关。绝对值小的负定点数会编码成环 $\llbracket p \rrbracket$ 上的大数，因此明文向量会包含许多大数，从而导致模型的噪声预算消耗增大，所以需要选择较小的 p 。同时为了使得解码得到正确结果，需要拥有较大的初始噪声预算，而初始噪声预算的大小与 q 正相关，因此需要选择较大的 q 。注意到 q 的上限与 N 正相关，因此也需要选择较大的 N 。
3. 在定点数编码下，每经过一次线性层的运算，密文 CNN 的中间结果相对于明文 CNN 的中间结果会扩大 $scale$ 倍，本文的 CNN 网络一共有 4 个线性层，因此密文 CNN 的输出向量会是明文 CNN 的输出向量的 $scale^5$ 。而定点数的绝对值不能超过 $\lfloor p/2 \rfloor$ ，因此需要选取较小的 $scale$ 和较大的 p 。
4. 明文 CNN 训练得到的模型参数多为小于 1 的浮点数，因此为了减小编码造成的误差，需要选择较大的 $scale$ 。而本实验采用的数据集较小，为

为了避免明文向量中存在大量内部碎片，需要选择较小的 N 。

注意到上述关系中存在许多矛盾，因此为了使得密文 CNN 具有高精确度和较高的运行效率，需要选择恰当的 scale, N, p, q 。下面通过检验由 N, p, q 生成的加密上下文，能否成功解密 $(p-1) \times [p-1]$ ，判断该组 (N, p, q) 是否满足密文 CNN 的需求。经过对 $N = 1024, 2048, 4096, 8192$ ，分别通过 SEAL 库里的 `Batching` 函数选择合适的 p ，并结合表 2.1 取最大的 q 进行实验，4.1 式中的 (N, p, q) 通过测试。

$$\begin{aligned} N &= 8192, p = 1099512004609, \\ q &= 8796092858369 \times 8796092792833 \times 17592186028033 \\ &\quad \times 17592185438209 \times 17592184717313. \end{aligned} \quad (4.1)$$

4.3.2 密文网络的实现

本小节基于 4.2 节的明文 CNN 结构和 4.3.1 节的加密参数，实现无截断层方案，并通过实验找到合适的 scale 。首先，密文 CNN 不需要进行反向传播，因而可以直接通过全连接层 2 的输出向量，得到推理结果，于是可以省略 softmax 层。接着对于 ReLU 层 1 与卷积层 2 以及 ReLU 层 2 与全连接层 2，分别使用 3.3.3 节和 3.3.6 节的协议进行优化，从而消除了卷积运算和矩阵乘法运算中的旋转操作，以及 3.2 节中的 Mux 操作。而对于卷积层 1 和全连接层 1，分别使用 3.3.2 节和 3.3.5 节的协议，同样消除了卷积运算和矩阵乘法运算中的旋转操作。最后需要确定缩放因子 scale ，注意到当 scale 过小时，编码后的模型参数损失较大，导致精确度降低；当 scale 过大时，中间结果数据溢出，导致精确度断崖式下降，因此并非 scale 越大越好。根据 4.3.1 节的参数关系得

$$\text{MAX_OUTPUT} \cdot \text{scale}^5 < \lfloor p/2 \rfloor \quad (4.2)$$

其中明文模数 $p = 1099512004609$ ，明文输出向量元素的上限 $\text{MAX_OUTPUT} < 20$ 。因此 scale 理论应当控制在 100 以内。

下面对于分别对 $\text{scale}=10, 20, 40, 80, 100, 110, 120, 130, 140, 150, 160$ 进行密文 CNN 推理实验，根据其推理准确率找到最合适的 scale 作为无截断层方案的缩放因子，实验结果如图 4.1 所示。根据实验结果，本文选取 $\text{scale} = 100$ 作为缩放因子，继续进行实验。

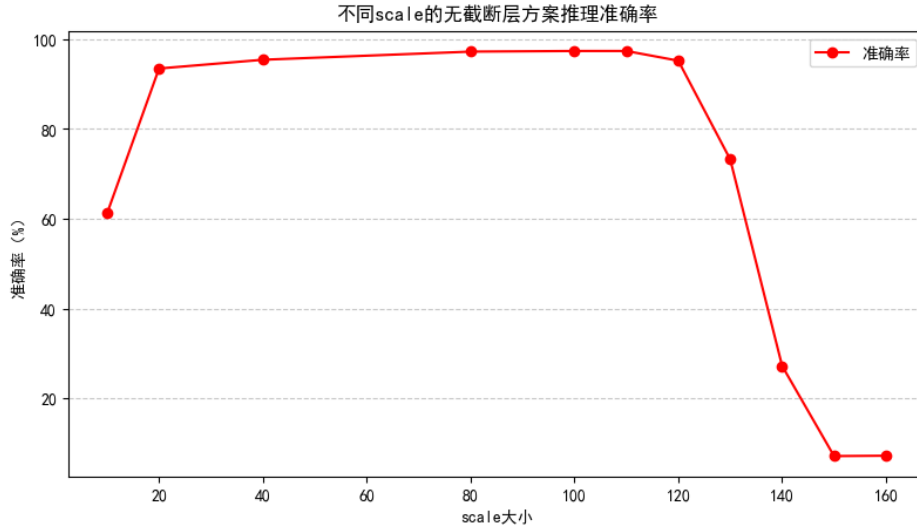


图 4.1 不同 scale 的无截断层方案推理准确率折线图

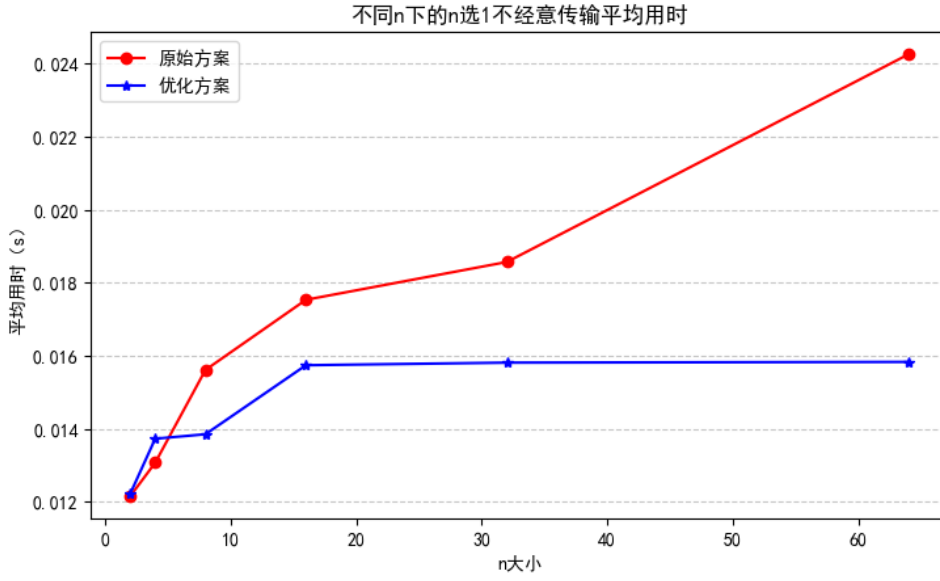
4.3.3 截断操作的引入

4.3.2 节中发现为了防止中间数据溢出，缩放因子应选取 100 左右，从而模型参数的精度只能保留小数点后两位。由此造成的误差，对于推理小型数据集 MNIST 的轻量 CNN，只会轻微影响推理结果。而对于推理大型数据集的深度神经网络，其线性层数远远大于本文的 CNN，因此其 scale 会远小于 100，从而每次计算线性层的误差会更大，进而多次计算线性层累计的误差会进一步增大，导致推理准确率大幅降低。

为了解决上述问题，下面引入截断操作，即在每次计算线性层之后，将中间结果除以 scale，从而消除了线性层中定点数乘法带来的数据膨胀，具体的截断操作通过 3.4 节的共享除法协议实现。通过对 scale=100、1000、10000、100000、1000000 进行推理实验，将密文 CNN 每个线性层的中间结果，与明文 CNN 每个线性层的中间结果进行对比，发现当 scale = 10000 时，中间误差值在 0.001 以内，拥有良好的推理准确率；当 scale = 100000 时，部分中间结果出现数据溢出，推理准确率略微下降；当 scale = 1000000 时，大量中间结果出现数据溢出，推理准确率大幅下降。因此选取 scale = 10000 作为带截断层方案的缩放因子。

4.3.4 $\binom{n}{1}$ -OT 协议的选择

本文中提出了两个 $\binom{n}{1}$ -OT 协议方案，分别在 2.5.2 节和 3.5 节中呈现。在理论上，虽然 3.5 节中的优化方案的时间复杂度为 $O(\log n)$ ，2.5.2 节中的原始方案

图 4.2 不同 n 下的 $\binom{n}{1}$ -OT 平均用时折线图

的时间复杂度为 $O(n)$ ，要低一个数量级，但优化方案还会增加一次通信开销。因此当 n 较大时，优化方案毫无疑问具有更高性能，但当 n 较小时，原始方案反而会有更好的效率，具体结果如图 4.2 所示。

本文中涉及了许多具有较小 n 的 $\binom{n}{1}$ -OT 协议。例如在 DReLU 协议、共享乘法协议和共享除法协议中， n 的取值通常为 4 或 8，所以在这些协议中使用原始方案。而在姚氏百万富翁问题的优化协议中，其调用的 $\binom{n}{1}$ -OT 具有较大的 n ，通常为 16、32 或 64，因此在该协议中采用优化方案。

第四节 性能对比分析

4.4.1 运行效率

本文对不同数量的测试用例，分别在明文网络、多项式逼近方案、无截断层方案和带截断层方案进行测试。图 4.3 展示了测试数量为 100、200、300、400、500、600、700、800、900、1000 时，上述三种方案处理单个测试用例的平均用时。

红色折线代表明文网络，其运行效率稳定在 0.005 秒/张左右。

蓝色折线代表多项式逼近方案，该方案对测试样例进行批处理，且编码的用例数量必须等于明文多项式度数 $N = 8192$ ，不足部分需要进行填充，因此其

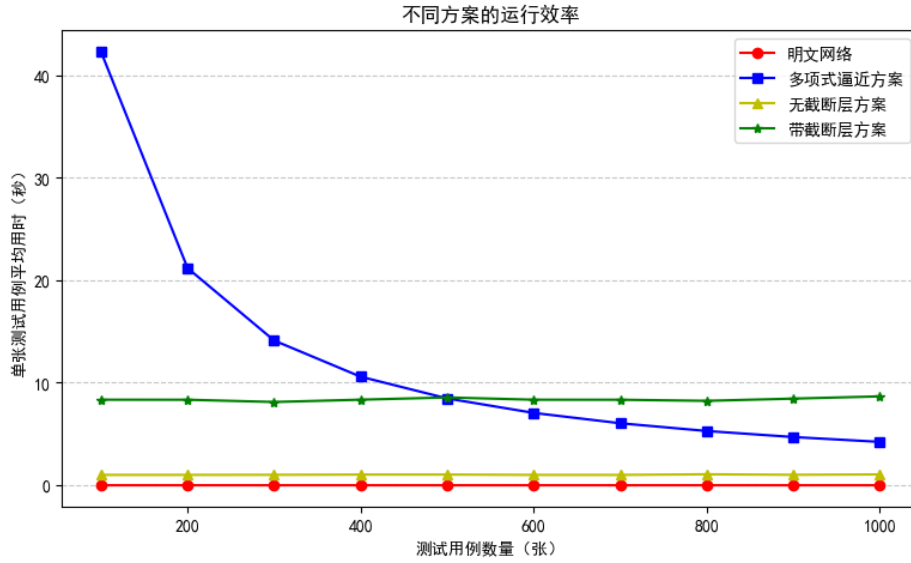


图 4.3 不同方案的运行效率折线图

运行效率与测试用例数量变化成反比。当测试用例数量接近 100 时，其运行效率约为 42 秒/张，其时间开销为明文网络的 8400 倍，该运行效率十分低下；当测试用例数量接近 1000 时，其运行效率为 4 秒/张，其时间开销为明文网络的 800 倍。

黄色折线代表无截断层方案，该方案利用 3.1 节、3.2 节和 3.3 节对密文网络进行优化，其运行效率稳定在 1 秒/张左右，约为多项式逼近方案在测试用例为 1000 时的 4 倍，其时间开销为明文网络的 200 倍。因此该方案在测试用例数量在 1000 左右时，相较于多项式逼近方案，有较大的运行效率优势，适用于大部分用户需求。

绿色折线代表带截断层方案，该方案在无截断层方案的基础上，结合 3.4 节对密文网络进行优化，其运行效率稳定在 8 秒/张左右，约为多项式逼近方案在测试用例数量为 200 时的 2.5 倍，其时间开销为明文网络的 1600 倍。其运行效率远低于无截断层方案的原因为该方案引入了四次截断操作，每次截断操作执行中包含四次 DReLU，导致其相较于执行 2 次 DReLU 的无截断层方案，额外增加了 16 次 DReLU 的计算。尽管其运行效率低于无截断层方案，但当测试用例数量为 100 ~ 200 等小规模时，相较于多项式逼近方案，同样有不小的运行效率优势，且其在精确度方面有较大优势，能适应复杂推理网络。

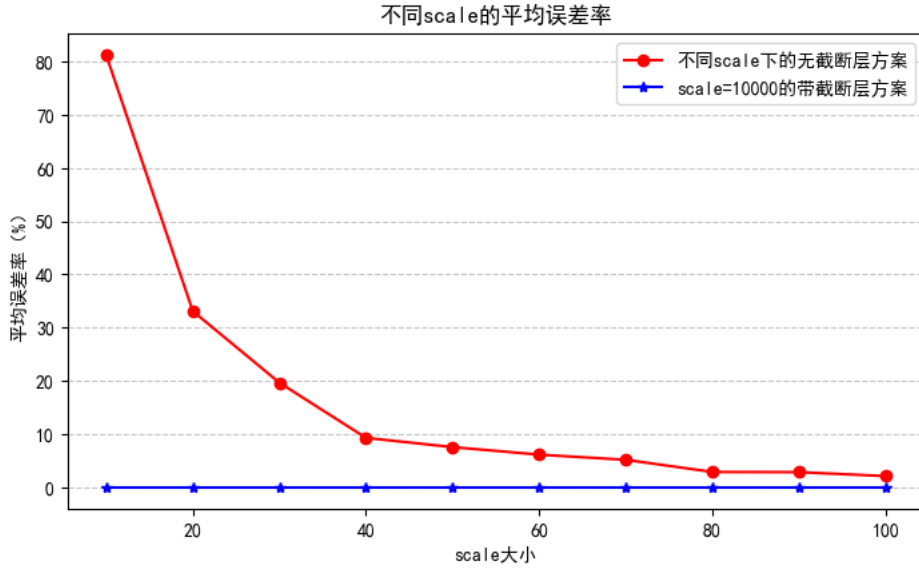


图 4.4 不同 scale 的平均误差率折线图

4.4.2 精确度

在定点数编码下，计算结果的精确度与缩放因子 $scale$ 有关。理论上，当密文网络所能支持的 $scale$ 越大，其精确度就越高。图 4.4 展示了无截断层方案在 $scale=10、20、30、40、50、60、70、90、100$ 时，以及带截断层方案在 $scale=10000$ 时，密文输出向量 $y \in \mathbb{R}^{10}$ 与明文输出向量 $x \in \mathbb{R}^{10}$ 的平均误差率 $\bar{\alpha}$ ，其中误差率 α 按 4.3 式定义。

$$\alpha = \left(\frac{\sum_{i=0}^9 (y_i - x_i)^2}{\sum_{i=0}^9 x_i^2} \right) \cdot 100\%. \quad (4.3)$$

通过图 4.4 可知，当无截断层方案的 $scale$ 在 $80 \sim 100$ 之间时，平均误差率在 2.5% 左右，从而可以推测其推理准确率也应更高。而带截断层方案在 $scale=10000$ 时，平均误差率仅有 0.00002%，从而可以推测其推理准确率与明文网络的推理准确率相同。下面进行了具体实验，图 4.5 展示了 $scale=20、30、40、50、60、70、90、100$ 时，多项式逼近方案和无截断层方案的密文网络推理准确率。

红色折线代表明文网络，其准确率为 97.40%。

蓝色折线代表多项式逼近方案，该方案涉及计算多项式函数，会引入大量密文与密文之间的乘法，从而快速消耗噪声预算。当噪声预算消耗完时，解密

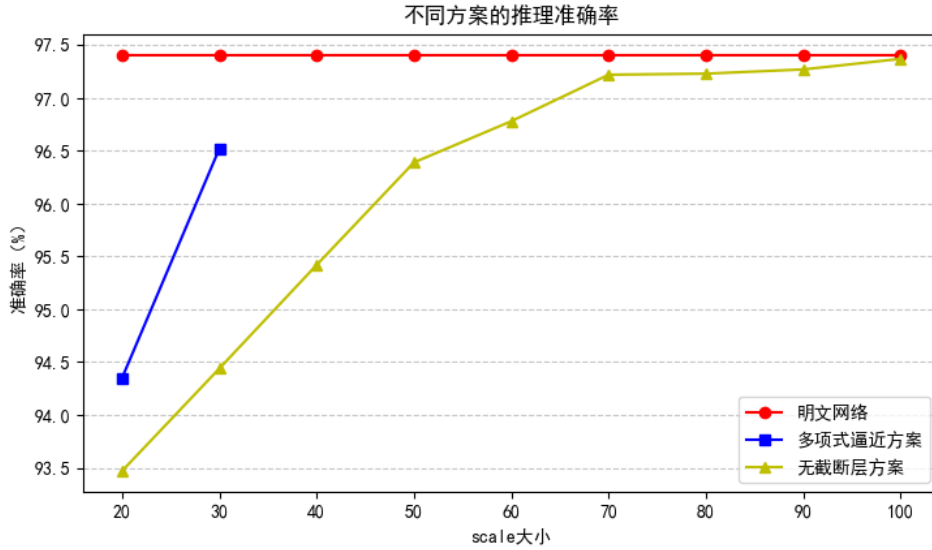


图 4.5 不同方案的推理准确率折线图

将不会得到正确的结果。并且通过多项式逼近 ReLU 函数，也会造成不可忽视的误差。当 ReLU 层数较多时，误差会积累到影响推理结果的程度。同时多项式函数会加剧数据膨胀问题，因此该方案只支持小 scale，而小 scale 会进一步导致误差增加。当 scale=30 时，其准确率为 96.51%；当 scale 大等于 40 时就会出现数据溢出问题，使得计算结果出现不可预测的错误导致最终结果，进而导致推理完全不准确。

黄色折线代表无截断层方案，该方案只涉及密文与明文的乘法，消耗噪声预算较慢，解密均可得到正确结果。且该方案精确计算 ReLU 函数，从而此处不会产生误差。同时该方案没有次方运算，因此其数据膨胀程度远小于多项式逼近方案，从而可以选择更大的 scale，进而其误差更小，推理准确率也更高。当 scale=100 时，其准确率为 97.36%，较多项式逼近方案有较大提升。

带截断层方案引入了截断操作，不会产生数据膨胀问题，因此 scale 的可选择的范围远大于上述两个方案。在本文中取 scale=10000 时，带截断层方案的推理准确率达到 97.40%，与明文网络相同。

4.4.3 实用性

第一，多项式逼近方案采用多项式函数逼近 ReLU 函数时，除了产生较大误差外，还会产生严重的数据膨胀，导致该方案只适用于小型 CNN，不能灵活添加新的运算层。而本文方案准确地实现了 ReLU 函数，没有理论误差，且不会

造成数据膨胀。除此之外，本文的带截断层方案，还消除了线性层的数据膨胀，从而支持灵活添加新的运算层，支持大型 CNN。

第二，多项式逼近方案的运行效率优势必须建立在大量测试用例上，当测试用例较少时，运行效率会大幅降低。而本文的无截断层方案并未对大量测试用例进行批处理，而是对单个测试用例的所有像素进行批处理编码，并实现了无旋卷积和无旋矩阵乘法运算，具有多项式逼近方案的线性层运算优势，同时对测试用例数量没有严格要求，支持用户灵活使用。

第五章 总结与展望

随着 MLaaS 广泛应用于各个领域，参与方愈加重视保护用户的数据隐私和服务器的模型知识产权，提出了 MLaaS 需要具有隐私保护能力的要求。本论文基于同态加密、不经意传输和秘密共享等安全多方计算技术，对推理网络进行加密。并在小型数据集 MNIST 上，实现了一个具有高准确度和高运行效率的分类 CNN 网络。

首先基于分治思想和安全多方计算技术，实现了具有高运行效率的姚氏百万富翁问题协议。接着为了实现具有高精确度的 ReLU 函数，本文基于 ReLU 函数与 DReLU 函数的关系，利用姚氏百万富翁问题协议实现了 DReLU 协议。最后为了消除在同态加密下卷积运算和矩阵乘法运算的旋转操作，并减少计算 ReLU 函数的通信开销和计算开销，对 CNN 的线性层和 ReLU 层进行联合优化。此外，本文实现了共享除法协议，并在 CNN 中引入截断层，成功避免了定点数编码的数据膨胀问题。

本文通过测试不同的加密参数对模型参数的适应性，确定了适合本文 CNN 的明文多项式度数、明文模数和密文模数。接着对定点数编码的缩放因子进行测试，确定了无截断层方案的最佳缩放因子。同时也引入截断层解决数据膨胀问题，实现了带截断层方案，并通过实验测试出其最佳缩放因子。最后在运行效率、精确度和实用性三个方面，对多项式逼近方案、无截断层优化方案和带截断层优化方案进行对比实验，发现本文的方案具有高运行效率、高精确度、高扩展性和高实用性。

未来主要的工作有如下两点：

1. 在无截断层方案中，DReLU 的时间开销最大，因此优化 DReLU 的通信开销和计算开销，或者利用其它安全多方计算技术实现更高效的 ReLU 函数，是提高运行效率的关键；
2. 在带截断层方案中，为了解决数据膨胀问题，引入截断操作的时间开销远大于无截断层方案的时间开销，因此未来可能通过更加巧妙的编码方式消除数据膨胀问题，或者实现更加高效且不失精确度的除法协议解决数据膨胀问题。

参考文献

- [1] Nathan Dowlin, Ran Gilad-Bachrach, Kim Laine, et al. CryptoNets: applying neural networks to encrypted data with high throughput and accuracy. International Conference on Machine Learning, International Conference on Machine Learning, June 2016.
- [2] 王南, 袁也, 杨浩然, et al. 环保大数据在区块链中的隐私计算. 信息安全, 2024, (10): 1515–1527.
- [3] Pratyush Mishra, Ryan Lehmkuhl, Akshayaram Srinivasan, et al. Delphi: A Cryptographic Inference Service for Neural Networks. USENIX Security Symposium, USENIX Security Symposium, Jan. 2020.
- [4] Y. Lecun, L. Bottou, Y. Bengio, et al. Gradient-based learning applied to document recognition. Proceedings of the IEEE, Jan. 1998: 2278–2324. <http://dx.doi.org/10.1109/5.726791>.
- [5] Alex Krizhevsky, Ilya Sutskever, Geoffrey E. Hinton. ImageNet Classification with Deep Convolutional Neural Networks. Communications of the ACM, May 2017: 84–90. <http://dx.doi.org/10.1145/3065386>.
- [6] Ronald L. Rivest, Michael L. Dertouzos. ON DATA BANKS AND PRIVACY HOMOMORPHISMS. Jan. 1978.
- [7] Joppe W. Bos, Kristin Lauter, Jake Loftus, et al. Improved Security for a Ring-Based Fully Homomorphic Encryption Scheme. IACR Cryptology ePrint Archive, IACR Cryptology ePrint Archive, Jan. 2013.
- [8] Florian Schaetz. BFV-complex and higher homotopy structures. arXiv: Quantum Algebra, arXiv: Quantum Algebra, Nov. 2006.
- [9] Ahmet Can Mert, Erdinc Ozturk, Erkan Savas. Design and Implementation of Encryption/Decryption Architectures for BFV Homomorphic Encryption Scheme. IEEE Transactions on Very Large Scale Integration (VLSI) Systems, Feb. 2020: 353–362. <http://dx.doi.org/10.1109/tvlsi.2019.2943127>.
- [10] Gilles Brassard, Claude Crepeau, Jean-Marc Robert. All-or-Nothing Disclosure of Secrets. In: Advances in Cryptology —CRYPTO’ 86, Lecture Notes in Com-

- puter Science, Apr. 2007: 234–238. http://dx.doi.org/10.1007/3-540-47721-7_17.
- [11] Moni Naor, Benny Pinkas. Efficient oblivious transfer protocols. Symposium on Discrete Algorithms, Symposium on Discrete Algorithms, Jan. 2001.
- [12] Yuval Ishai, Joe Kilian, Kobbi Nissim, et al. Extending oblivious transfers efficiently. In: Advances in Cryptology - CRYPTO 2003, Lecture Notes in Computer Science, Jan. 2003: 145–161. http://dx.doi.org/10.1007/978-3-540-45146-4_9.
- [13] Adi Shamir. How to share a secret. Communications of the ACM, Nov. 1979: 612–613. <http://dx.doi.org/10.1145/359168.359176>.
- [14] Donald Beaver. Efficient Multiparty Protocols Using Circuit Randomization. In: Advances in Cryptology —CRYPTO ’91, Lecture Notes in Computer Science, Aug. 2007: 420–432. http://dx.doi.org/10.1007/3-540-46766-1_34.
- [15] Andrew Chi-Chih Yao. Protocols for secure computations. Foundations of Computer Science, Foundations of Computer Science, Nov. 1982.
- [16] Geoffroy Couteau. New Protocols for Secure Equality Test and Comparison. IACR Cryptology ePrint Archive, IACR Cryptology ePrint Archive, Jan. 2016.
- [17] 李顺东, 戴一奇, 游启友. 姚氏百万富翁问题的高效解决方案. 电子学报, 2005, 33(5): 5.
- [18] Juan Garay, Berry Schoenmakers, José Villegas. Practical and Secure Solutions for Integer Comparison. In: Public Key Cryptography –PKC 2007, Lecture Notes in Computer Science, June 2007: 330–342. http://dx.doi.org/10.1007/978-3-540-71677-8_22.
- [19] Payman Mohassel, Peter Rindal. ABY 3: A Mixed Protocol Framework for Machine Learning.
- [20] Chiraag Juvekar, Vinod Vaikuntanathan, Anantha P. Chandrakasan. GAZELLE: A Low Latency Framework for Secure Neural Network Inference. USENIX Security Symposium, USENIX Security Symposium, Jan. 2018.
- [21] Fabian Boemer, Rosario Cammarota, Daniel Demmler, et al. MP2ML. In: Proceedings of the 15th International Conference on Availability, Reliability and Security, Aug. 2020. <http://dx.doi.org/10.1145/3407023.3407045>.

- [22] Payman Mohassel, Yupeng Zhang. SecureML: A System for Scalable Privacy-Preserving Machine Learning. In: 2017 IEEE Symposium on Security and Privacy (SP), May 2017. <http://dx.doi.org/10.1109/sp.2017.12>.
- [23] Zhicong Huang, Wen-Jie Lu, Cheng Hong, et al. Cheetah: Lean and Fast Secure Two-Party Deep Neural Network Inference.
- [24] Deevashwer Rathee, Mayank Rathee, Nishant Kumar, et al. CrypTFlow2: Practical 2-Party Secure Inference. In: Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security, Oct. 2020. <http://dx.doi.org/10.1145/3372297.3417274>.

致 谢

行文至此，意味着我的本科生涯即将画上句点。回首这段求学时光，心中充满感激之情。在此，我谨向所有给予我支持、帮助和鼓励的人致以最诚挚的谢意。

首先，衷心感谢我的指导老师苏明老师。从论文选题、框架设计到细节修改，您始终以严谨的学术态度和耐心的指导为我指明方向。您渊博的学识、敏锐的学术洞察力以及对待学生的包容与鼓励，让我受益匪浅。

感谢南开大学计算机学院的全体授课老师，是你们在专业课程中的悉心传授，为我奠定了扎实的理论基础。同时，感谢辅导员谭彤彤老师在学业与生活中的关怀，让我在迷茫时找到前行的动力。

同窗之谊，一生珍藏。感谢我的室友和同学们，特别是华志远、孔德嵘等好友，与你们一起探讨学术、分享生活的日子是我青春中最珍贵的回忆。感谢师兄师姐杨浩然、许友锐在论文写作中的经验分享，让我少走了许多弯路。

最后，深深感恩我的家人。你们无私的爱与支持是我求学路上最坚实的后盾。无论顺境逆境，你们始终是最温暖的港湾。

寥寥数语，难表谢意。未来我将以更饱满的热情迎接挑战，不负所期。